Program No: 12	Date: 09/1/2026

Program Title: Write a web application using JavaScript and HTML that takes a city name as input, fetches weather data from a public API, processes the JSON response, and displays temperature, weather condition, humidity, and wind speed.

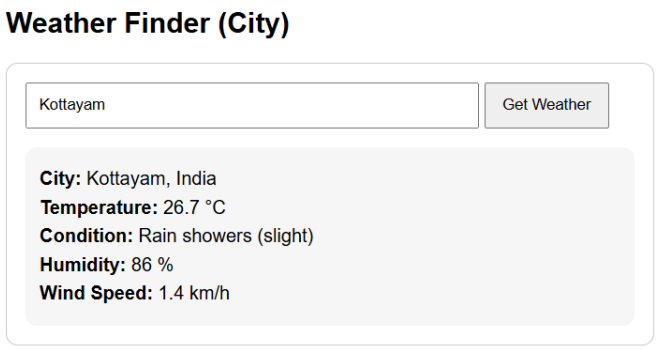
```
<!—Weather web application
@Ann Jo Mathew
Roll No: 12
09/1/2026-->

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
  <title>City Weather App</title>
  <style>
    body { font-family: Arial, sans-serif; padding: 20px; }
    .box { max-width: 520px; padding: 16px; border: 1px solid #ccc; border-radius: 10px; }
    input { width: 70%; padding: 10px; }
    button { padding: 10px 14px; cursor: pointer; }
    .result { margin-top: 16px; padding: 12px; border-radius: 10px; background: #f6f6f6; }
    .error { color: #b00020; margin-top: 10px; }
    .row { margin: 6px 0; }
    .label { font-weight: bold; }
  </style>
</head>
<body>

  <h2>Weather Finder (City)</h2>

  <div class="box">
    <input id="cityInput" type="text" placeholder="Enter city name (e.g., Kochi)" />
    <button id="btn">Get Weather</button>

    <div id="msg" class="error"></div>

    <div id="output" class="result" style="display:none;">
      <div class="row"><span class="label">City:</span> <span id="cityOut"></span></div>
      <div class="row"><span class="label">Temperature:</span> <span
id="tempOut"></span></div>
      <div class="row"><span class="label">Condition:</span> <span
id="condOut"></span></div>
      <div class="row"><span class="label">Humidity:</span> <span
id="humOut"></span></div>
      <div class="row"><span class="label">Wind Speed:</span> <span
id="windOut"></span></div>
```

```
  </div>
</div>

<script>
  const cityInput = document.getElementById("cityInput");
  const btn = document.getElementById("btn");
  const msg = document.getElementById("msg");
  const output = document.getElementById("output");

  const cityOut = document.getElementById("cityOut");
  const tempOut = document.getElementById("tempOut");
  const condOut = document.getElementById("condOut");
  const humOut = document.getElementById("humOut");
  const windOut = document.getElementById("windOut");

  // Convert Open-Meteo weather code to a readable condition
  function weatherCodeToText(code) {
    const map = {
      0: "Clear sky",
      1: "Mainly clear",
      2: "Partly cloudy",
      3: "Overcast",
      45: "Fog",
      48: "Depositing rime fog",
      51: "Light drizzle",
      53: "Moderate drizzle",
      55: "Dense drizzle",
      56: "Freezing drizzle (light)",
      57: "Freezing drizzle (dense)",
      61: "Slight rain",
      63: "Moderate rain",
      65: "Heavy rain",
      66: "Freezing rain (light)",
      67: "Freezing rain (heavy)",
      71: "Slight snow fall",
      73: "Moderate snow fall",
      75: "Heavy snow fall",
      77: "Snow grains",
      80: "Rain showers (slight)",
      81: "Rain showers (moderate)",
      82: "Rain showers (violent)",
      85: "Snow showers (slight)",
      86: "Snow showers (heavy)",
      95: "Thunderstorm",
      96: "Thunderstorm with slight hail",
      99: "Thunderstorm with heavy hail"
    };
```

```
    return map[code] ?? "Unknown";
  }

  async function getWeather() {
    msg.textContent = "";
    output.style.display = "none";

    const city = cityInput.value.trim();
    if (!city) {
      msg.textContent = "Enter a city name.";
      return;
    }

    try {
      // 1) Geocoding: city -> latitude/longitude
      const geoUrl = `https://geocoding-api.open-
meteo.com/v1/search?name=${encodeURIComponent(city)}&count=1`;
      const geoRes = await fetch(geoUrl);
      if (!geoRes.ok) throw new Error("Geocoding request failed.");

      const geoData = await geoRes.json();
      if (!geoData.results || geoData.results.length === 0) {
        msg.textContent = "City not found. Try another name.";
        return;
      }

      const place = geoData.results[0];
      const lat = place.latitude;
      const lon = place.longitude;

      // 2) Weather: fetch current weather + humidity + wind
      // current=temperature_2m, relative_humidity_2m, weather_code,
wind_speed_10m
      const weatherUrl =
        `https://api.open-meteo.com/v1/forecast?latitude=${lat}&longitude=${lon}` +

`&current=temperature_2m,relative_humidity_2m,weather_code,wind_speed_10m` +
        `&timezone=auto`;

      const wRes = await fetch(weatherUrl);
      if (!wRes.ok) throw new Error("Weather request failed.");

      const wData = await wRes.json();

      // 3) Process JSON and display
      const cur = wData.current;
```

```
      cityOut.textContent = `${place.name}${place.country ? ", " + place.country : ""}`;
      tempOut.textContent = `${cur.temperature_2m}
${wData.current_units.temperature_2m}`;
      condOut.textContent = weatherCodeToText(cur.weather_code);
      humOut.textContent = `${cur.relative_humidity_2m}
${wData.current_units.relative_humidity_2m}`;
      windOut.textContent = `${cur.wind_speed_10m}
${wData.current_units.wind_speed_10m}`;

      output.style.display = "block";
    } catch (err) {
      msg.textContent = "Error: " + err.message;
    }
  }

  btn.addEventListener("click", getWeather);
  cityInput.addEventListener("keydown", (e) => {
    if (e.key === "Enter") getWeather();
  });
 </script>
</body>
</html>
```

Output



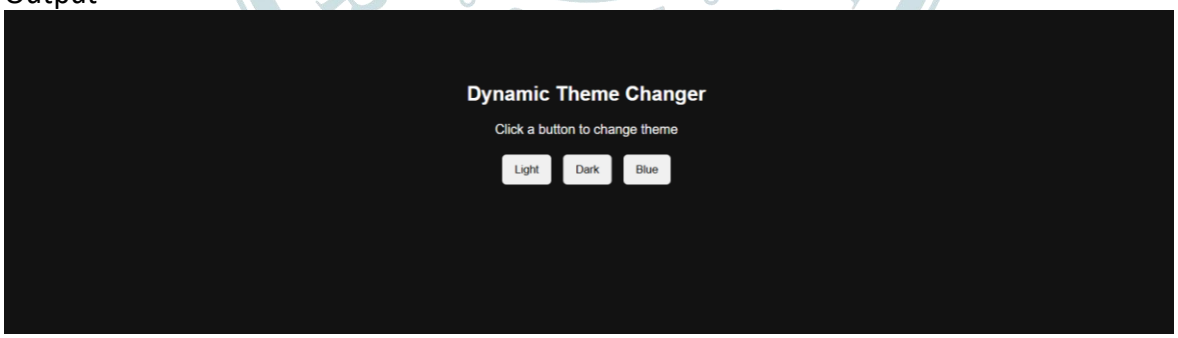
Program No: 13	Date: 11/01/2026

Program Title: Create a webpage that changes theme dynamically using JavaScript.

```
<!—Dynamic theme change
@Ann Jo Mathew
Roll No: 12
07/12/2026 -->

<!DOCTYPE html>
<html>
<head>
  <title>Dynamic Theme Changer</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      transition: 0.4s;
    }

    .container {
      text-align: center;
      margin-top: 100px;
    }

    button {
      padding: 10px 15px;
      margin: 5px;
      cursor: pointer;
      border: none;
      border-radius: 5px;
    }

    /* Default Light Theme */
    .light {
      background-color: white;
      color: black;
    }

    /* Dark Theme */
    .dark {
      background-color: #121212;
      color: white;
    }

    /* Blue Theme */
    .blue {
```

```
        background-color: #1e3a8a;
        color: white;
      }
    </style>
</head>

<body class="light">

    <div class="container">
      <h2>Dynamic Theme Changer</h2>
      <p>Click a button to change theme</p>

      <button onclick="setTheme('light')">Light</button>
      <button onclick="setTheme('dark')">Dark</button>
      <button onclick="setTheme('blue')">Blue</button>
    </div>

    <script>
      function setTheme(themeName) {
        document.body.className = themeName;
      }
    </script>

</body>
</html>
```

Output



Program No: 14	Date: 09/1/2026

Program Title: Design a responsive webpage using Bootstrap 5 grid system with Bootstrap-based login and registration page.

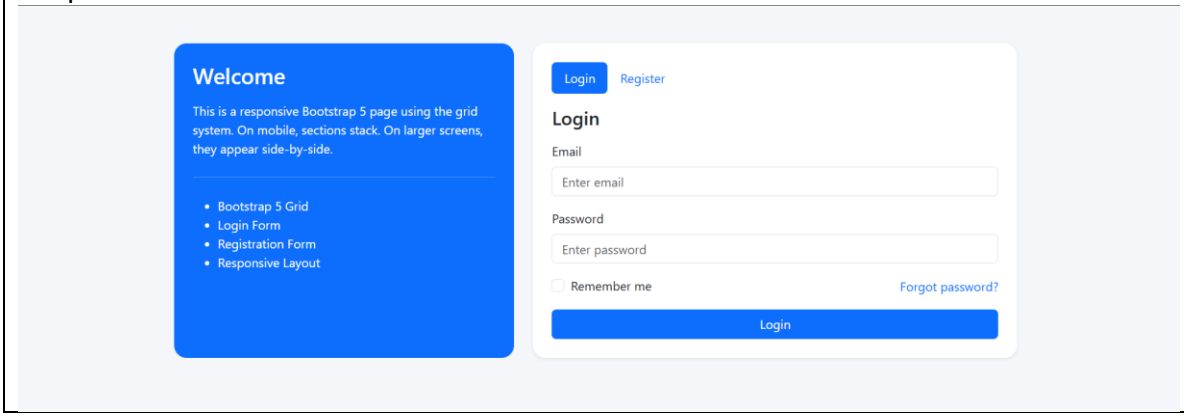
```
<!—Webpage using Bootstrap 5 grid system
@Ann Jo Mathew
Roll No: 12
Date: 12/01/2026 -->

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Bootstrap 5 Login & Registration</title>

  <!-- Bootstrap 5 CSS -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">

  <style>
    body { background: #f4f6f9; }
    .auth-card { border: 0; border-radius: 16px; }
  </style>
</head>
<body>

  <div class="container py-5">
    <div class="row justify-content-center">
      <!-- Main responsive grid: left info + right forms -->
      <div class="col-12 col-lg-10">
        <div class="row g-4 align-items-stretch">

          <!-- Left side (Info) -->
          <div class="col-12 col-md-5">
            <div class="h-100 p-4 text-white rounded-4" style="background: #0d6efd;">
              <h3 class="mb-3">Welcome</h3>
              <p class="mb-0">
                This is a responsive Bootstrap 5 page using the grid system.
                On mobile, sections stack. On larger screens, they appear side-by-side.
              </p>
              <hr class="my-4">
              <ul class="mb-0">
                <li>Bootstrap 5 Grid</li>
                <li>Login Form</li>
                <li>Registration Form</li>
```

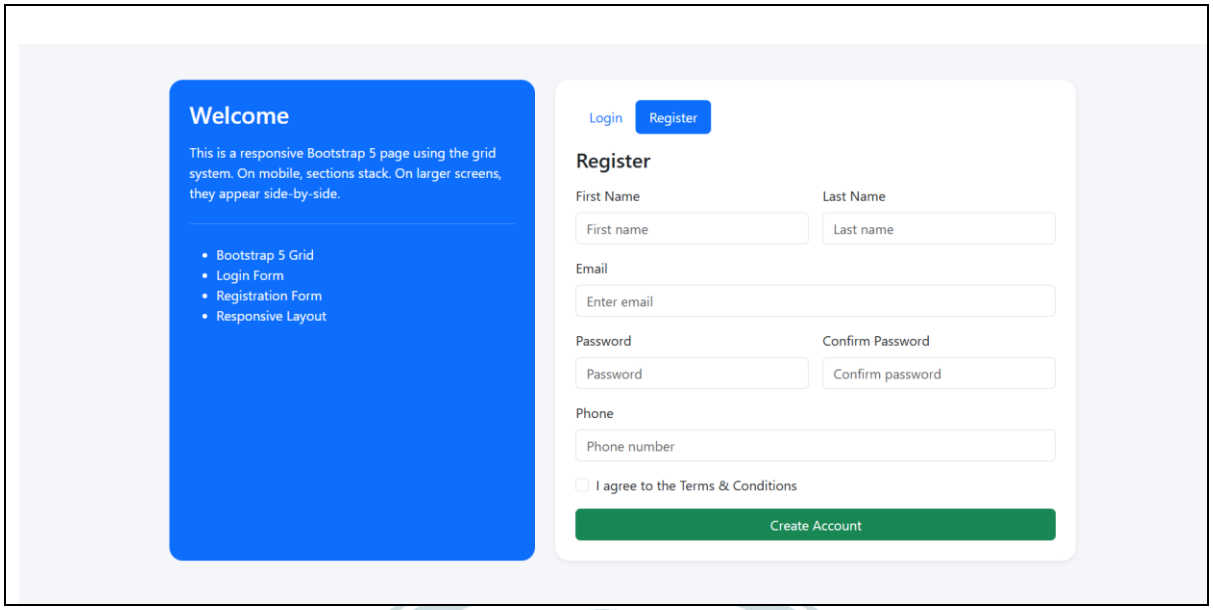
```
            <li>Responsive Layout</li>
          </ul>
        </div>
      </div>

      <!-- Right side (Forms) -->
      <div class="col-12 col-md-7">
        <div class="card auth-card shadow-sm h-100">
          <div class="card-body p-4">

            <!-- Tabs -->
            <ul class="nav nav-pills mb-3" id="authTabs" role="tablist">
              <li class="nav-item" role="presentation">
                <button class="nav-link active" id="login-tab" data-bs-toggle="pill" data-bs-
target="#login"
                  type="button" role="tab">Login</button>
              </li>
              <li class="nav-item" role="presentation">
                <button class="nav-link" id="register-tab" data-bs-toggle="pill" data-bs-
target="#register"
                  type="button" role="tab">Register</button>
              </li>
            </ul>

            <!-- Tab Content -->
            <div class="tab-content">

              <!-- Login Form -->
              <div class="tab-pane fade show active" id="login" role="tabpanel">
                <h4 class="mb-3">Login</h4>
                <form>
                  <div class="mb-3">
                    <label class="form-label">Email</label>
                    <input type="email" class="form-control" placeholder="Enter email"
required>
                  </div>

                  <div class="mb-3">
                    <label class="form-label">Password</label>
                    <input type="password" class="form-control" placeholder="Enter password"
required>
                  </div>

                  <div class="d-flex justify-content-between align-items-center mb-3">
                    <div class="form-check">
                      <input class="form-check-input" type="checkbox" id="rememberMe">
```

```
                <label class="form-check-label" for="rememberMe">Remember
me</label>
              </div>
              <a href="#" class="text-decoration-none">Forgot password?</a>
            </div>

            <button type="submit" class="btn btn-primary w-100">Login</button>
          </form>
        </div>

        <!-- Registration Form -->
        <div class="tab-pane fade" id="register" role="tabpanel">
          <h4 class="mb-3">Register</h4>
          <form>
            <div class="row g-3">
              <div class="col-12 col-md-6">
                <label class="form-label">First Name</label>
                <input type="text" class="form-control" placeholder="First name"
required>
              </div>
              <div class="col-12 col-md-6">
                <label class="form-label">Last Name</label>
                <input type="text" class="form-control" placeholder="Last name"
required>
              </div>
            </div>

            <div class="mb-3 mt-3">
              <label class="form-label">Email</label>
              <input type="email" class="form-control" placeholder="Enter email"
required>
            </div>

            <div class="row g-3">
              <div class="col-12 col-md-6">
                <label class="form-label">Password</label>
                <input type="password" class="form-control" placeholder="Password"
required>
              </div>
              <div class="col-12 col-md-6">
                <label class="form-label">Confirm Password</label>
                <input type="password" class="form-control" placeholder="Confirm
password" required>
              </div>
            </div>

            <div class="mb-3 mt-3">
```

```
                <label class="form-label">Phone</label>
                <input type="tel" class="form-control" placeholder="Phone number"
required>
              </div>

              <div class="form-check mb-3">
                <input class="form-check-input" type="checkbox" id="terms" required>
                <label class="form-check-label" for="terms">
                  I agree to the Terms & Conditions
                </label>
              </div>

              <button type="submit" class="btn btn-success w-100">Create
Account</button>
            </form>
          </div>

        </div><!-- tab-content -->

      </div><!-- card-body -->
    </div><!-- card -->
  </div>

  </div><!-- row -->
 </div><!-- col -->
 </div><!-- row -->
</div><!-- container -->

<!-- Bootstrap 5 JS -->
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></sc
ript>
</body>
</html>
```

Output




Welcome
This is a responsive Bootstrap 5 page using the grid system. On mobile, sections stack. On larger screens, they appear side-by-side.
Bootstrap 5 Grid
Login Form
Registration Form
Responsive Layout
Login
Register
Register
First Name
Last Name
First name
Last name
Email
Enter email
Password
Confirm Password
Password
Confirm password
Phone
Phone number
I agree to the Terms & Conditions
Create Account

Program No: 15	Date: 09/01/2026

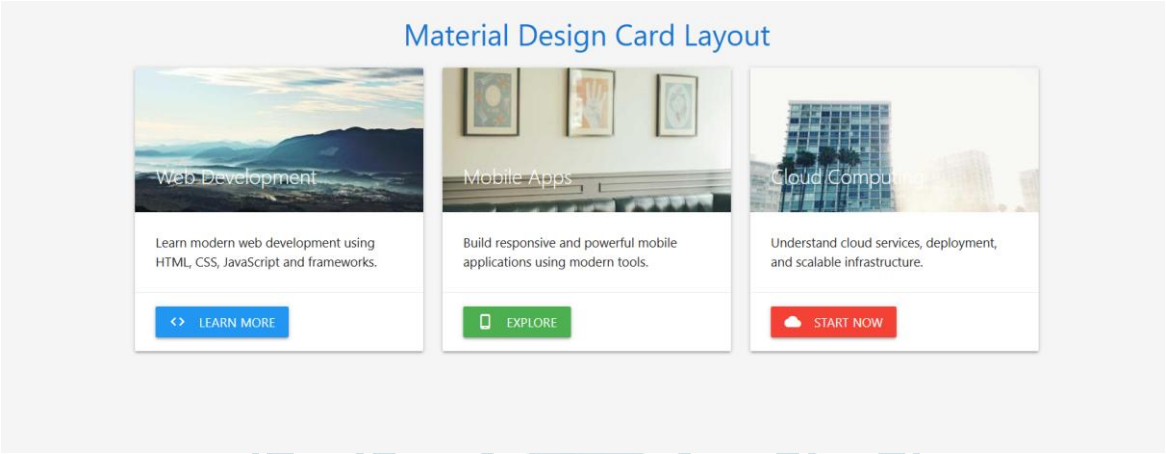
Program Title: Design a webpage using Material Design cards and buttons.

```
<!—Webpage using Material design cards and buttons
@Ann Jo Mathew
Roll No: 12
14/01/2026 -->

<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Material Design Cards</title>

  <!-- Google Icons -->
  <link href="https://fonts.googleapis.com/icon?family=Material+Icons" rel="stylesheet">

  <!-- Materialize CSS -->
  <link
href="https://cdnjs.cloudflare.com/ajax/libs/materialize/1.0.0/css/materialize.min.css"
rel="stylesheet">

  <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
</head>

<body class="grey lighten-4">

  <div class="container">
    <h4 class="center-align blue-text text-darken-2">Material Design Card Layout</h4>

    <div class="row">

      <!-- Card 1 -->
      <div class="col s12 m6 l4">
        <div class="card">
          <div class="card-image">
            <img src="https://picsum.photos/400/200?1">
            <span class="card-title">Web Development</span>
          </div>
          <div class="card-content">
            <p>Learn modern web development using HTML, CSS, JavaScript and
frameworks.</p>
          </div>
          <div class="card-action">
            <a class="btn waves-effect waves-light blue">
```

```
          <i class="material-icons left">code</i>Learn More
        </a>
      </div>
    </div>
  </div>

  <!-- Card 2 -->
  <div class="col s12 m6 l4">
    <div class="card">
      <div class="card-image">
        <img src="https://picsum.photos/400/200?2">
        <span class="card-title">Mobile Apps</span>
      </div>
      <div class="card-content">
        <p>Build responsive and powerful mobile applications using modern tools.</p>
      </div>
      <div class="card-action">
        <a class="btn waves-effect waves-light green">
          <i class="material-icons left">phone_android</i>Explore
        </a>
      </div>
    </div>
  </div>

  <!-- Card 3 -->
  <div class="col s12 m6 l4">
    <div class="card">
      <div class="card-image">
        <img src="https://picsum.photos/400/200?3">
        <span class="card-title">Cloud Computing</span>
      </div>
      <div class="card-content">
        <p>Understand cloud services, deployment, and scalable infrastructure.</p>
      </div>
      <div class="card-action">
        <a class="btn waves-effect waves-light red">
          <i class="material-icons left">cloud</i>Start Now
        </a>
      </div>
    </div>
  </div>

  </div>
</div>

<!-- Materialize JS -->
```

```
 <script
src="https://cdnjs.cloudflare.com/ajax/libs/materialize/1.0.0/js/materialize.min.js"></scri
pt>

</body>
</html>
```

Output



Program No: 16	Date: 16/01/2026

Program Title: Combine Bootstrap and Material Design for a professional UI.

```
<!—Bootstrap and Material Design
@Ann Jo Mathew
Roll No: 12
18/01/2026 -->

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
  <title>Bootstrap + Material Design UI</title>

  <!-- Bootstrap 5 CSS (layout + grid) -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">

  <!-- Material Symbols (optional icons) -->
  <link href="https://fonts.googleapis.com/css2?family=Material+Symbols+Outlined"
rel="stylesheet" />

  <style>
    body { background: #f5f7fb; }
    .topbar {
      background: white;
      border-bottom: 1px solid #e9ecef;
    }
    .hero {
      background: linear-gradient(135deg, #0d6efd, #6f42c1);
      color: white;
      border-radius: 18px;
    }
    .section-title { font-weight: 700; }
    .muted { color: #6c757d; }
    .material-symbols-outlined { vertical-align: middle; font-size: 20px; }
  </style>

  <!-- Material Web Components (M3) -->
  <script type="module"
src="https://cdn.jsdelivr.net/npm/@material/web/all.js"></script>
</head>

<body>
```

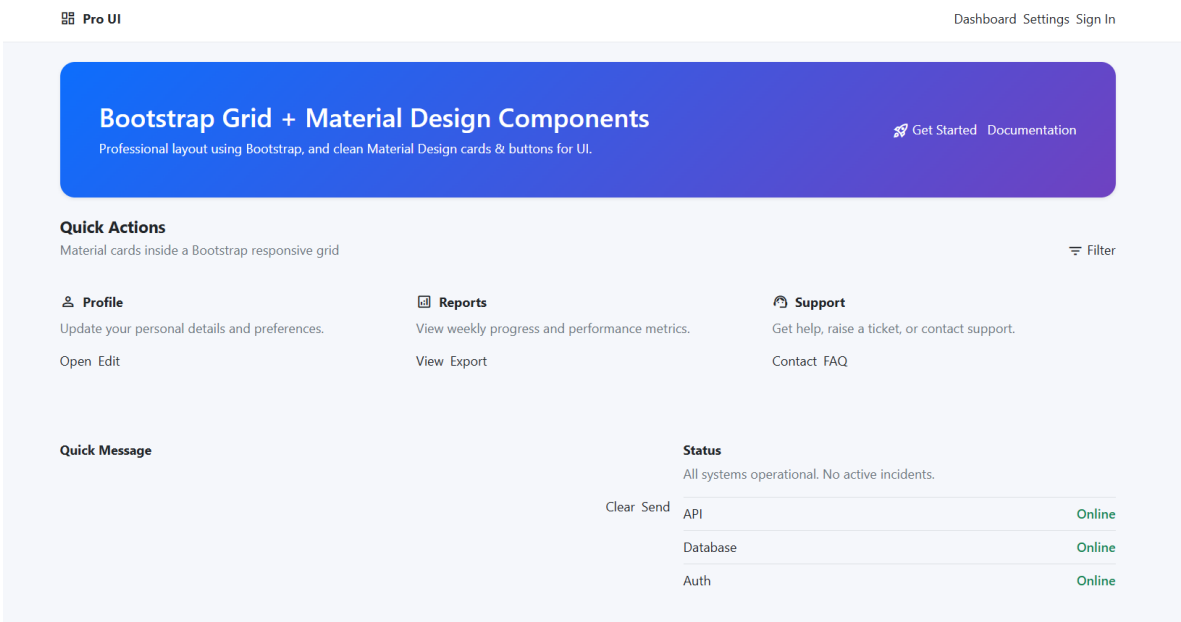
```
<!-- Topbar (Bootstrap layout) -->
<div class="topbar">
  <div class="container py-3 d-flex align-items-center justify-content-between">
    <div class="d-flex align-items-center gap-2">
      <span class="material-symbols-outlined">dashboard</span>
      <strong>Pro UI</strong>
    </div>

    <div class="d-flex align-items-center gap-2">
      <!-- Material buttons -->
      <md-filled-tonal-button>Dashboard</md-filled-tonal-button>
      <md-outlined-button>Settings</md-outlined-button>
      <md-filled-button>Sign In</md-filled-button>
    </div>
  </div>
</div>

<!-- Main Content -->
<div class="container py-4">

  <!-- Hero (Bootstrap grid + spacing) -->
  <div class="hero p-4 p-md-5 mb-4 shadow-sm">
    <div class="row align-items-center g-4">
      <div class="col-12 col-md-7">
        <h2 class="mb-2">Bootstrap Grid + Material Design Components</h2>
        <p class="mb-0">
          Professional layout using Bootstrap, and clean Material Design cards & buttons for
UI.
        </p>
      </div>
      <div class="col-12 col-md-5 text-md-end">
        <md-filled-button>
          <span class="material-symbols-outlined" slot="icon">rocket_launch</span>
          Get Started
        </md-filled-button>
        <md-outlined-button class="ms-2">Documentation</md-outlined-button>
      </div>
    </div>
  </div>

  <!-- Cards Section -->
  <div class="d-flex align-items-end justify-content-between mb-3">
    <div>
      <div class="section-title h5 mb-1">Quick Actions</div>
      <div class="muted">Material cards inside a Bootstrap responsive grid</div>
    </div>
```

```
  <md-outlined-button>
    <span class="material-symbols-outlined" slot="icon">filter_list</span>
    Filter
  </md-outlined-button>
</div>

<!-- Bootstrap grid holding Material cards -->
<div class="row g-3">
  <div class="col-12 col-md-6 col-lg-4">
    <md-elevated-card style="padding:16px; border-radius:16px;">
      <div class="d-flex align-items-center gap-2 mb-2">
        <span class="material-symbols-outlined">person</span>
        <strong>Profile</strong>
      </div>
      <p class="muted mb-3">Update your personal details and preferences.</p>
      <div class="d-flex gap-2">
        <md-filled-button>Open</md-filled-button>
        <md-outlined-button>Edit</md-outlined-button>
      </div>
    </md-elevated-card>
  </div>

  <div class="col-12 col-md-6 col-lg-4">
    <md-elevated-card style="padding:16px; border-radius:16px;">
      <div class="d-flex align-items-center gap-2 mb-2">
        <span class="material-symbols-outlined">analytics</span>
        <strong>Reports</strong>
      </div>
      <p class="muted mb-3">View weekly progress and performance metrics.</p>
      <div class="d-flex gap-2">
        <md-filled-button>View</md-filled-button>
        <md-outlined-button>Export</md-outlined-button>
      </div>
    </md-elevated-card>
  </div>

  <div class="col-12 col-md-6 col-lg-4">
    <md-elevated-card style="padding:16px; border-radius:16px;">
      <div class="d-flex align-items-center gap-2 mb-2">
        <span class="material-symbols-outlined">support_agent</span>
        <strong>Support</strong>
      </div>
      <p class="muted mb-3">Get help, raise a ticket, or contact support.</p>
      <div class="d-flex gap-2">
        <md-filled-button>Contact</md-filled-button>
        <md-outlined-button>FAQ</md-outlined-button>
      </div>
```

```
      </md-elevated-card>
    </div>
  </div>

  <!-- Bootstrap + Material form area -->
  <div class="row g-3 mt-4">
    <div class="col-12 col-lg-7">
      <md-elevated-card style="padding:16px; border-radius:16px;">
        <div class="section-title h6 mb-3">Quick Message</div>

        <!-- Material text fields -->
        <div class="row g-3">
          <div class="col-12 col-md-6">
            <md-outlined-text-field label="Name" style="width:100%;"></md-outlined-text-
field>
          </div>
          <div class="col-12 col-md-6">
            <md-outlined-text-field label="Email" type="email" style="width:100%;"></md-
outlined-text-field>
          </div>
          <div class="col-12">
            <md-outlined-text-field
              label="Message"
              type="text"
              style="width:100%;"
              textarea
              rows="4"></md-outlined-text-field>
          </div>
          <div class="col-12 d-flex justify-content-end gap-2">
            <md-outlined-button>Clear</md-outlined-button>
            <md-filled-button>Send</md-filled-button>
          </div>
        </div>
      </md-elevated-card>
    </div>

    <div class="col-12 col-lg-5">
      <md-elevated-card style="padding:16px; border-radius:16px;">
        <div class="section-title h6 mb-2">Status</div>
        <p class="muted mb-3">All systems operational. No active incidents.</p>

        <div class="d-flex align-items-center justify-content-between py-2 border-top">
          <span>API</span>
          <span class="text-success fw-semibold">Online</span>
        </div>
        <div class="d-flex align-items-center justify-content-between py-2 border-top">
          <span>Database</span>
```

```
        <span class="text-success fw-semibold">Online</span>
      </div>
      <div class="d-flex align-items-center justify-content-between py-2 border-top">
        <span>Auth</span>
        <span class="text-success fw-semibold">Online</span>
      </div>
    </md-elevated-card>
  </div>
</div>

</div>

<!-- Bootstrap JS (optional, if you use dropdowns/modals etc.) -->
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></sc
ript>
</body>
</html>
```

Output



Program No: 17	Date: 12/12/2025

Program Title: Assignment Requirements:

- Use Bootstrap 5 for layout and responsive design.
- Use at least three Material Design components (Cards, Buttons, Forms, Shadows,
- etc.).
- Implement Login and Registration pages for user and admin with proper validation.
- Use LocalStorage for all database operations (Create, Read, Update, Delete).
- Dynamically render products using JavaScript.

Include the following pages:

- Home Page
- Login Page
- Registration Page
- Product Listing Page
- Add Product Page
- Edit Product Page
- Cart Page

```
<!—Bootstrap 5 Shopping Webpage
@Ann Jo Mathew
Roll No: 12
16/12/2025 -->
```

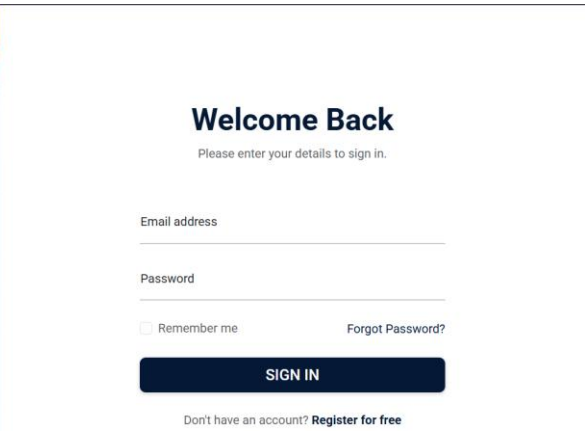
edit-product.html

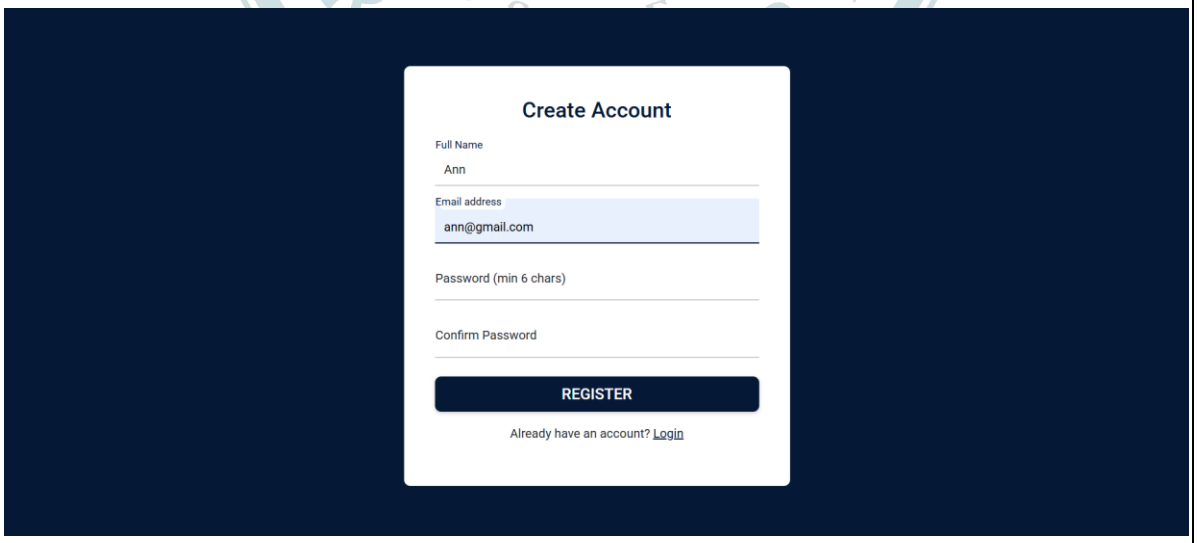
```
<!DOCTYPE html>
<html lang="en">

<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Edit Product - OmniShop</title>
   <!-- Bootstrap 5 CSS -->
   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet">
   <!-- Google Fonts -->
   <link
href="https://fonts.googleapis.com/css2?family=Roboto:wght@300;400;500;700&display
=swap" rel="stylesheet">
   <!-- Custom CSS -->
   <link rel="stylesheet" href="css/style.css">
</head>

<body>

   <div id="navbar-container"></div>

   <div class="container my-5">
```

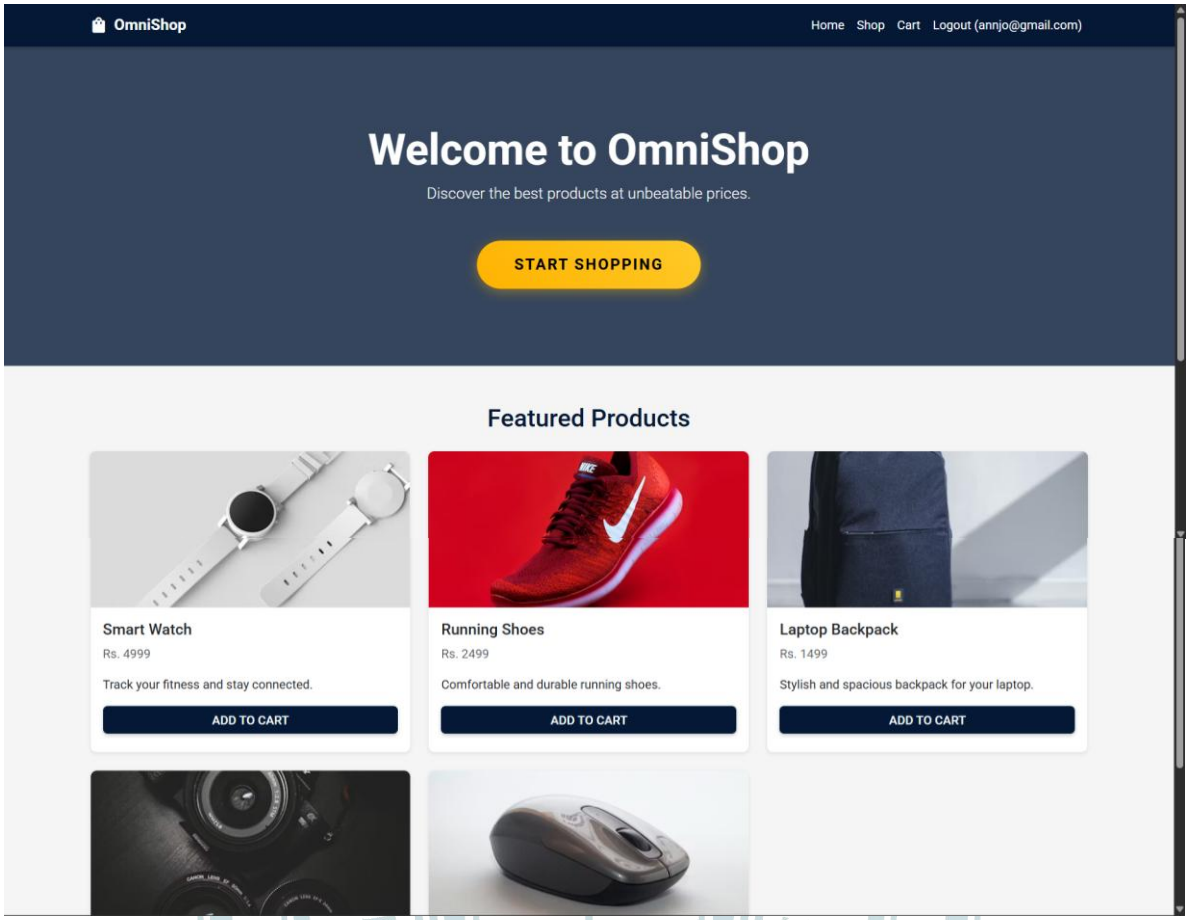
```
      <div class="row justify-content-center">
        <div class="col-md-8">
          <div class="card p-4">
            <h3 class="text-center mb-4" style="color: var(--primary-color);">Edit
Product</h3>
            <form onsubmit="editProduct(event)">
              <input type="hidden" id="productId">
              <div class="form-floating mb-3">
                <input type="text" class="form-control" id="name" placeholder="Product
Name" required>
                <label for="name">Product Name</label>
              </div>
              <div class="form-floating mb-3">
                <input type="number" class="form-control" id="price"
placeholder="Price" required min="0"
                  step="0.01">
                <label for="price">Price (Rs.)</label>
              </div>
              <div class="form-floating mb-3">
                <textarea class="form-control" placeholder="Description"
id="description"
                  style="height: 100px" required></textarea>
                <label for="description">Description</label>
              </div>
              <div class="form-floating mb-4">
                <input type="url" class="form-control" id="image" placeholder="Image
URL">
                <label for="image">Image URL</label>
              </div>
              <div class="d-grid gap-2">
                <button type="submit" class="btn btn-primary btn-lg">Update
Product</button>
              </div>
            </form>
          </div>
        </div>
      </div>
    </div>

    <div id="snackbar"></div>

    <!-- Scripts -->
    <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"></sc
ript>
    <script src="js/utils.js"></script>
    <script src="js/products.js"></script>
```

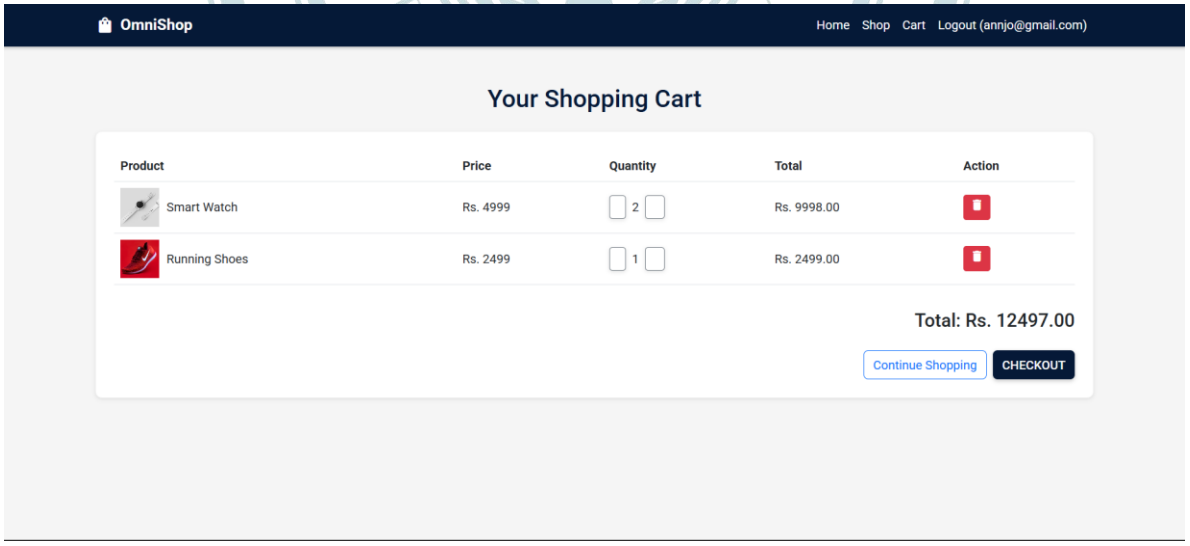
```
  <script>
    renderNavbar();
    // Protect Route
    if (getCurrentUser() !== 'admin@gmail.com') {
      window.location.href = 'index.html';
    }
    loadProductForEdit();
  </script>
</body>

</html>
```

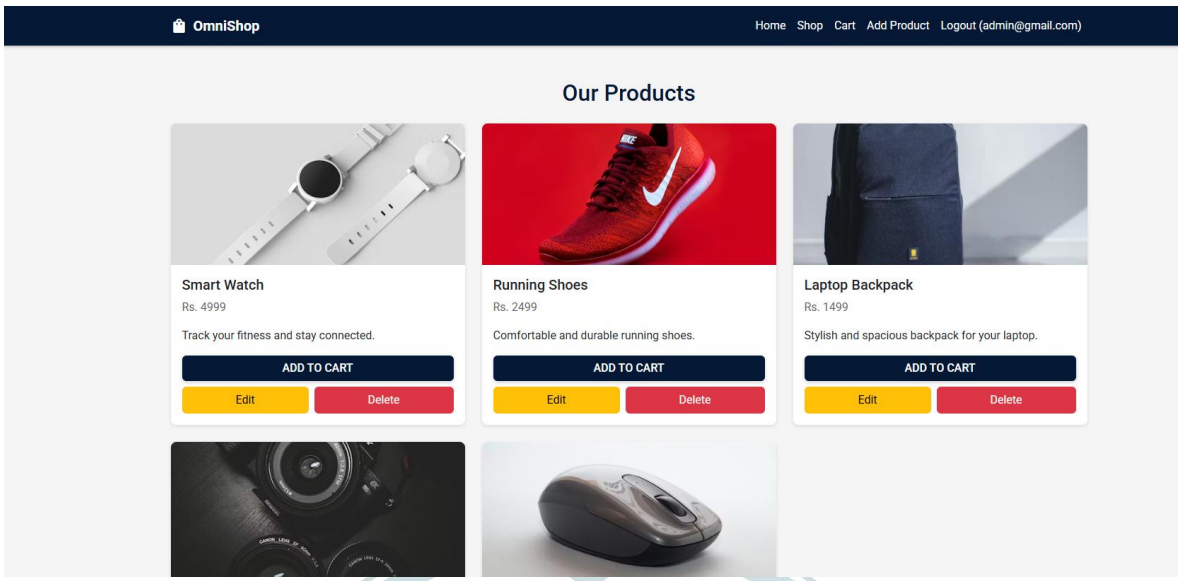
Output

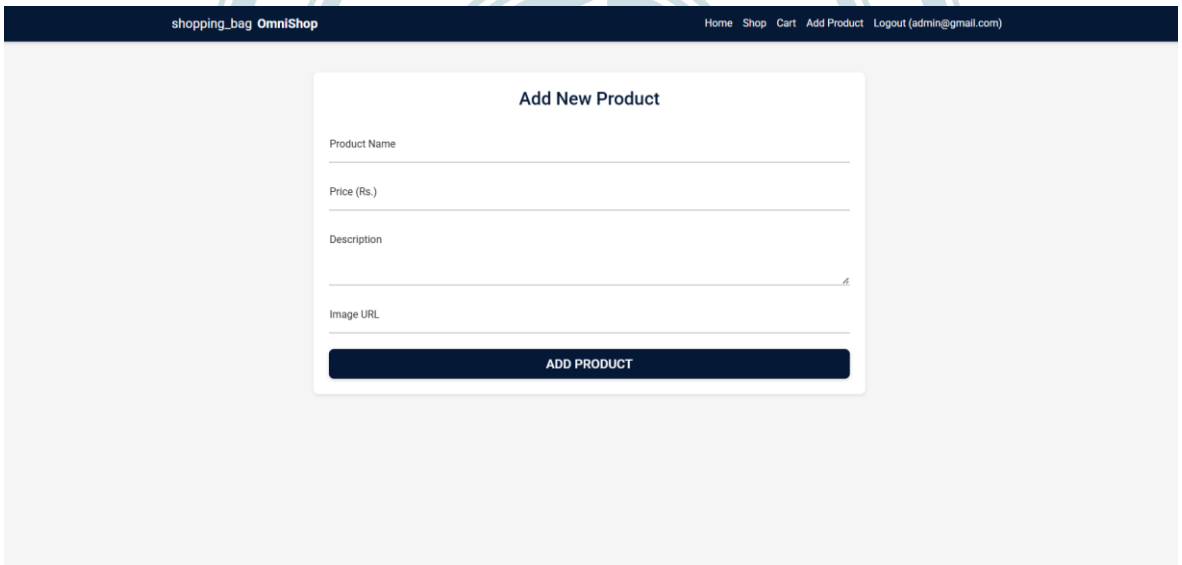





OmniShop
Home Shop Cart Logout (annjo@gmail.com)
Welcome to OmniShop
Discover the best products at unbeatable prices.
START SHOPPING
Featured Products
Smart Watch
Rs. 4999
Track your fitness and stay connected.
ADD TO CART
Running Shoes
Rs. 2499
Comfortable and durable running shoes.
ADD TO CART
Laptop Backpack
Rs. 1499
Stylish and spacious backpack for your laptop.
ADD TO CART


OmniShop
Home Shop Cart Logout (annjo@gmail.com)
Your Shopping Cart
Product
Price
Quantity
Total
Action
Smart Watch
Rs. 4999
2
Rs. 9998.00
Running Shoes
Rs. 2499
1
Rs. 2499.00
Total: Rs. 12497.00
Continue Shopping
CHECKOUT


OmniShop
Home Shop Cart Add Product Logout (admin@gmail.com)
Our Products
Smart Watch
Rs. 4999
Track your fitness and stay connected.
ADD TO CART
Edit
Delete
Running Shoes
Rs. 2499
Comfortable and durable running shoes.
ADD TO CART
Edit
Delete
Laptop Backpack
Rs. 1499
Stylish and spacious backpack for your laptop.
ADD TO CART
Edit
Delete


shopping_bag OmniShop
Home Shop Cart Add Product Logout (admin@gmail.com)
Add New Product
Product Name
Price (Rs.)
Description
Image URL
ADD PRODUCT

Program No: 18	Date: 12/12/2025

Program Title: Create a calculator in flask using ajax.

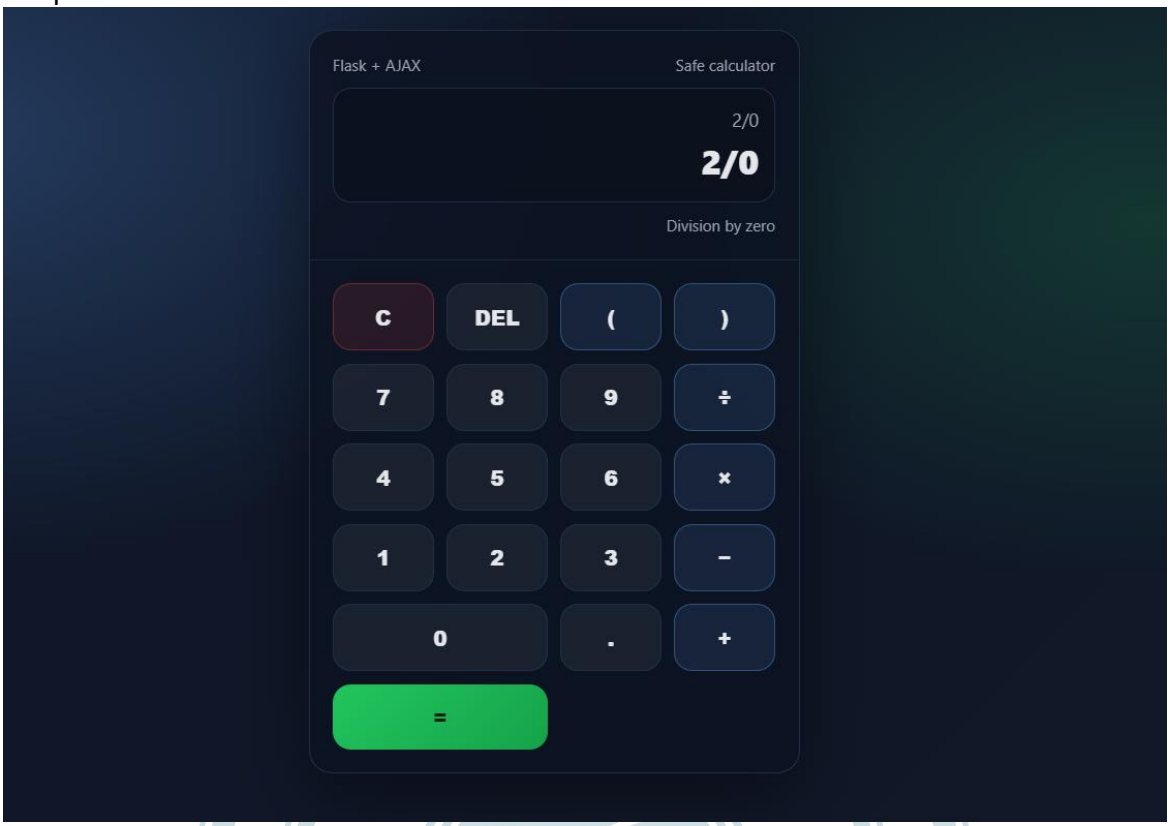
```
<!—Calculator using flask
@Ann Jo Mathew
Roll No: 12
16/12/2025 -->
```

app.py

```
from flask import Flask, render_template, request, jsonify
import ast
import operator as op

app = Flask(__name__)

# Allowed operations
OPS = {
    ast.Add: op.add,
    ast.Sub: op.sub,
    ast.Mult: op.mul,
    ast.Div: op.truediv,
    ast.USub: op.neg,
    ast.UAdd: op.pos,
}

def safe_eval(expr: str) -> float:
    """
    Safely evaluate math expressions with + - * / parentheses and decimals.
    Rejects everything else.
    """
    expr = expr.strip()
    if not expr:
        raise ValueError("Empty expression")

    node = ast.parse(expr, mode="eval")

    def _eval(n):
        if isinstance(n, ast.Expression):
            return _eval(n.body)

        # numbers
        if isinstance(n, ast.Constant) and isinstance(n.value, (int, float)):
            return n.value
```

```
        # unary +/-
        if isinstance(n, ast.UnaryOp) and type(n.op) in OPS:
            return OPS[type(n.op)](_eval(n.operand))

        # binary ops
        if isinstance(n, ast.BinOp) and type(n.op) in OPS:
            left = _eval(n.left)
            right = _eval(n.right)
            if isinstance(n.op, ast.Div) and right == 0:
                raise ZeroDivisionError("Division by zero")
            return OPS[type(n.op)](left, right)

        raise ValueError("Invalid expression")

    return _eval(node)

@app.get("/")
def home():
    return render_template("index.html")

@app.post("/calculate")
def calculate():
    data = request.get_json(silent=True) or {}
    expr = data.get("expr", "")

    try:
        result = safe_eval(expr)
        return jsonify({"ok": True, "result": result})
    except ZeroDivisionError:
        return jsonify({"ok": False, "error": "Division by zero"}), 400
    except Exception:
        return jsonify({"ok": False, "error": "Invalid expression"}), 400

if __name__ == "__main__":
    app.run(debug=True)
```

Output


Flask + AJAX
Safe calculator
2/0
2/0
Division by zero
C
DEL
(
)
7
8
9
÷
4
5
6
×
1
2
3
−
0
.
+
=

Program No: 19	Date: 12/12/2025

Program Title: Create a suggestion form using ajax in flask.

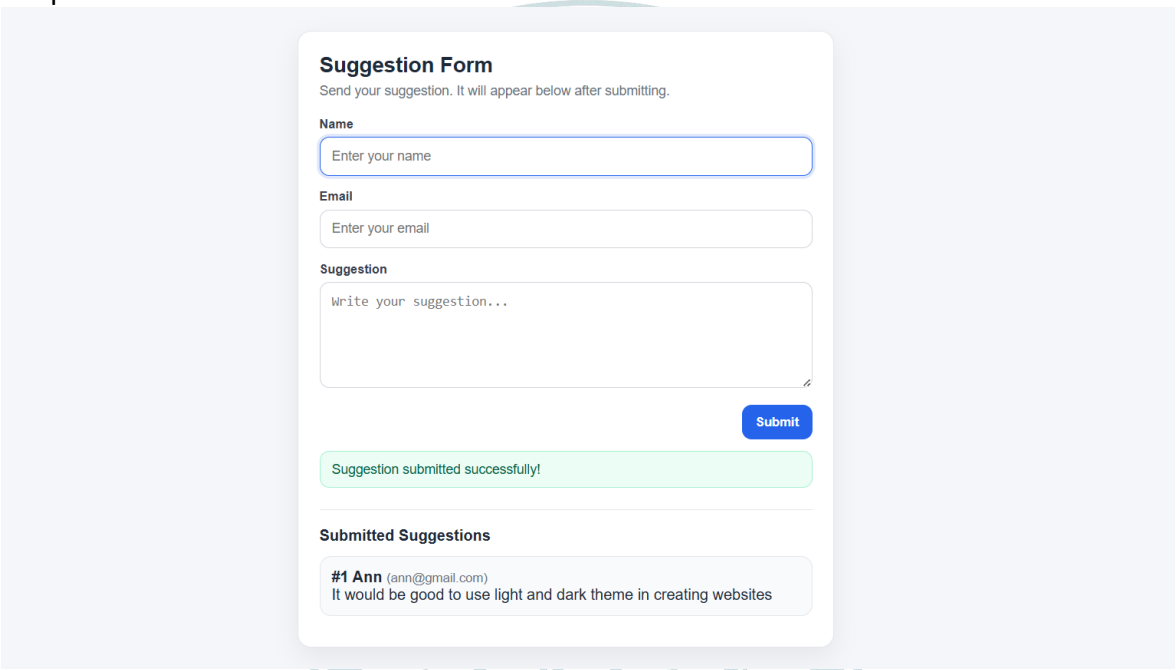
```
<!—Suggestion form in flask
@Ann Jo Mathew
Roll No: 12
19/12/2025 -->
```

app.py

```
from flask import Flask, render_template, request, jsonify

app = Flask(__name__)

# In-memory storage (for demo). Resets when server restarts.
SUGGESTIONS = []

@app.get("/")
def home():
    return render_template("index.html")

@app.post("/api/suggestions")
def save_suggestion():
    # Accept JSON from AJAX
    data = request.get_json(silent=True) or {}

    name = (data.get("name") or "").strip()
    email = (data.get("email") or "").strip()
    message = (data.get("message") or "").strip()

    # Basic validation
    if not name:
        return jsonify({"ok": False, "error": "Name is required."}), 400
    if not email or "@" not in email:
        return jsonify({"ok": False, "error": "Valid email is required."}), 400
    if not message or len(message) < 5:
        return jsonify({"ok": False, "error": "Suggestion must be at least 5 characters."}), 400

    # "Create" operation (store)
    item = {
        "id": len(SUGGESTIONS) + 1,
        "name": name,
        "email": email,
        "message": message
    }
```

```
    SUGGESTIONS.append(item)

    return jsonify({"ok": True, "msg": "Suggestion submitted successfully!", "item": item}),
201

@app.get("/api/suggestions")
def list_suggestions():
    # "Read" operation (optional)
    return jsonify({"ok": True, "items": SUGGESTIONS})

if __name__ == "__main__":
    app.run(debug=True)
```

Output



Program No: 20	Date: 19/12/2025

Program Title: Create a Django project and display Hello World on the browser.

```
<!—Hello World in Django
@Ann Jo Mathew
Roll No: 12
19/12/2025 -->
```

views.py

```
from django.http import HttpResponse

def home(request):
    return HttpResponse("App configured successfully!.")
```

myapp/urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.home),
]
```

myproject/urls.py

```
from django.contrib import admin
from django.urls import path,include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('',include('myapp.urls')),
]
```

Output

Hello World 👋 This page is rendered using Django.

Program No: 21	Date: 19/12/2025

Program Title: Create a Django app and configure it in the project.

```
<!-- Dropdown-based content rendering using JavaScript
@Ann Jo Mathew
Roll No: 12
21/12/2025 -->
```

settings.py

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',

    'myapp',
]
```

views.py

```
from django.http import HttpResponse
def home(request):
    return HttpResponse("App configured successfully!")
```

myapp/urls.py

```
from django.urls import path
from . import views
urlpatterns = [
    path('', views.home),
]
```

myproject/urls.py

```
from django.contrib import admin
from django.urls import path,include
urlpatterns = [
    path('admin/', admin.site.urls),
    path('',include('myapp.urls')),
]
```

Output

Program No: 22	Date: 19/12/2025

Program Title: Create a Django model for Student and perform migrations. Register models in the Django admin panel.

```
<!—Django models.py and admin panel
@Ann Jo Mathew
Roll No: 12
21/12/2025 -->
```

myapp/models.py

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()
    email = models.EmailField()

    def __str__(self):
        return self.name
```

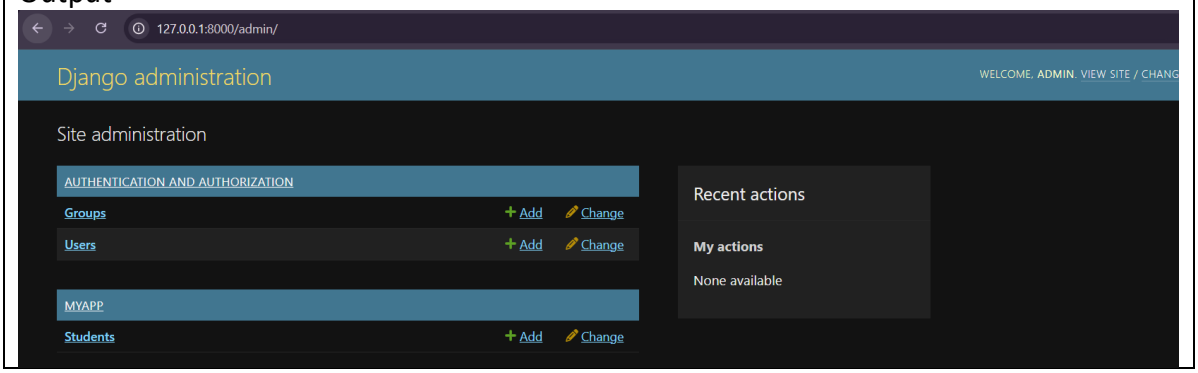
myapp/admin.py

```
from django.contrib import admin
from .models import Student

admin.site.register(Student)
```

myproject/urls.py

```
from django.contrib import admin
from django.urls import path,include
urlpatterns = [
    path('admin/', admin.site.urls),
    path('',include('myapp.urls')),
]
```

Output



Program No: 23	Date: 12/12/2025

Program Title: Perform CRUD operations using Django ORM.

```
<!—CRUD operations using Django ORM
@Ann Jo Mathew
Roll No: 12
19/12/2025 -->
```

myapp/views.py

```
from django.shortcuts import render, redirect
from django.http import HttpResponse
from .models import Student

# READ (List all students)
def student_list(request):
    students = Student.objects.all()
    return render(request, "student_list.html", {"students": students})

# CREATE (Add new student)
def student_create(request):
    if request.method == "POST":
        name = request.POST.get("name")
        age = request.POST.get("age")
        email = request.POST.get("email")
        Student.objects.create(name=name, age=age, email=email)
        return redirect("student_list")
    return render(request, "student_create.html")

# UPDATE (Edit existing student)
def student_update(request, id):
    student = Student.objects.get(id=id)
    if request.method == "POST":
        student.name = request.POST.get("name")
        student.age = request.POST.get("age")
        student.email = request.POST.get("email")
        student.save()
        return redirect("student_list")
    return render(request, "student_update.html", {"student": student})

# DELETE (Delete a student)
def student_delete(request, id):
    student = Student.objects.get(id=id)
    if request.method == "POST":
        student.delete()
        return redirect("student_list")
    return render(request, "student_delete.html", {"student": student})
```

myapp/templates/student_list.html

```
<h2>Students</h2>
<a href="{% url 'student_create' %}">+ Add Student</a>
<hr>
  <table border="1" cellpadding="8">
    <tr><th>Name</th><th>Age</th><th>Email</th><th>Actions</th></tr>
    {% for s in students %}
    <tr><td>{{ s.name }}</td><td>{{ s.age }}</td><td>{{ s.email }}</td>
      <td>
        <a href="{% url 'student_update' s.id %}">Edit</a> |
        <a href="{% url 'student_delete' s.id %}">Delete</a>
      </td>
    </tr>
    {% endfor %}
  </table>
```

myapp/templates/ student_create.html

```
<h2>Add Student</h2>
<form method="POST">
{% csrf_token %}
Name: <input type="text" name="name" required><br><br>
Age: <input type="number" name="age" required><br><br>
Email: <input type="email" name="email" required><br><br>
<button type="submit">Save</button>
</form><br>
<a href="{% url 'student_list' %}">Back</a>
```

myapp/templates/ student_update.html

```
<h2>Edit Student</h2>
<form method="POST">
{% csrf_token %}
Name: <input type="text" name="name" value="{{ student.name }}" required><br><br>
Age: <input type="number" name="age" value="{{ student.age }}" required><br><br>
Email: <input type="email" name="email" value="{{ student.email }}" required><br><br>
<button type="submit">Update</button>
</form><br>
<a href="{% url 'student_list' %}">Back</a>
```

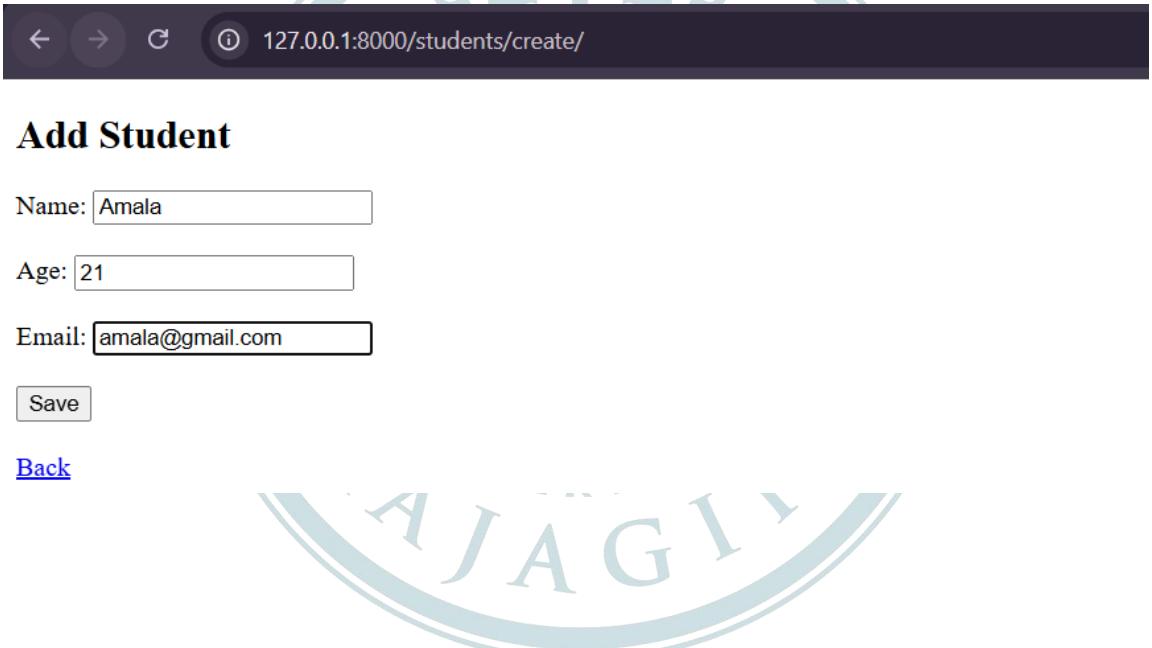
myapp/templates/student_delete.html

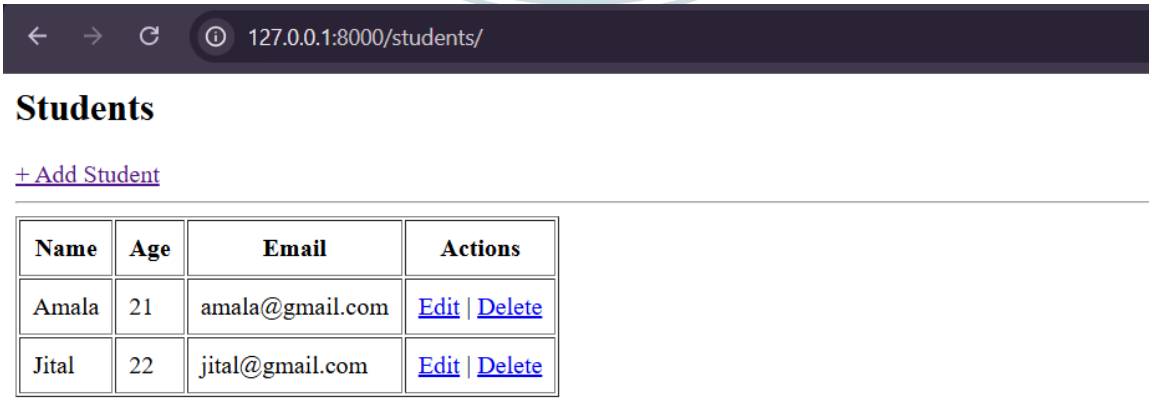
```
<h2>Delete Student</h2>
<p>Are you sure you want to delete <b>{{ student.name }}</b>?</p>
<form method="POST">
{% csrf_token %}
<button type="submit">Yes, Delete</button>
</form><br>
<a href="{% url 'student_list' %}">Cancel</a>
```

myapp/urls.py

```
from django.urls import path
from . import views

urlpatterns = [
   path("students/", views.student_list, name="student_list"),              # READ
   path("students/create/", views.student_create, name="student_create"),     # CREATE
   path("students/edit/<int:id>/", views.student_update, name="student_update"), # UPDATE
   path("students/delete/<int:id>/", views.student_delete, name="student_delete"), # DELETE
]
```

Output





127.0.0.1:8000/students/edit/2/

Edit Student

Name: Jital

Age: 21

Email: jital@gmail.com

Update

Back

127.0.0.1:8000/students/delete/2/

Delete Student

Are you sure you want to delete **Jital**?

Yes, Delete

Cancel

Program No: 24	Date: 19/12/2025

Program Title: Create function-based views in Django.

```
<!—Function-based views
@Ann Jo Mathew
Roll No: 12
19/12/2025 -->
```

myapp/views.py

```
from django.shortcuts import render
from .models import Student

# Function Based View (FBV)
def student_list(request):
    students = Student.objects.all()   # READ from database
    return render(request, 'list.html', {'students': students})
```

myapp/urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path('students/', views.student_list, name='students'),
]
```

myapp/templates/list.html

```
<h2>Student List</h2>
    {% for student in students %}
        <p>
            Name: {{ student.name }} |
            Age: {{ student.age }} |
            Email: {{ student.email }}
        </p>
    {% endfor %}
```

Output

Student List

Name: Amala | Age: 21 | Email: amala@gmail.com

Program No: 25	Date: 19/12/2025

Program Title: Create class-based views in Django.

```
<!—class-based views
@Ann Jo Mathew
Roll No: 12
21/12/2025 -->
```

myapp/views.py

```
from django.views.generic import ListView
from .models import Student
# Class-Based View
class StudentListView(ListView):
    model = Student
    template_name = "list.html"
    context_object_name = "students"
```

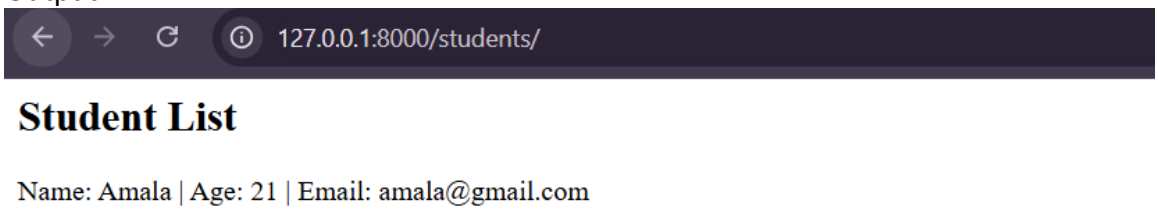
myapp/urls.py

```
from django.urls import path
from .views import StudentListView
urlpatterns = [
    path('students/', StudentListView.as_view(), name='students'),
]
```

templates/list.html

```
<h2>Student List (Class Based View)</h2>
    {% for student in students %}
        <p>
            Name: {{ student.name }} |
            Age: {{ student.age }} |
            Email: {{ student.email }}
        </p>
    {% endfor %}
```

Output



Program No: 26	Date: 19/12/2025

Program Title: Configure dynamic URL routing in Django.

```
<!—Dynamic URL routing
@Ann Jo Mathew
Roll No: 12
21/12/2025 -->
```

app/urls.py

```
from django.urls import path
from . import views
urlpatterns = [
   path("students/", views.student_list, name="student_list"),            # normal
   path("students/<int:id>/", views.student_detail, name="student_detail"), # dynamic
]
```

app/views.py

```
from django.shortcuts import render, get_object_or_404
from .models import Student
def student_list(request):
   students = Student.objects.all()
   return render(request, "list.html", {"students": students})
def student_detail(request, id):
   student = get_object_or_404(Student, id=id)
   return render(request, "detail.html", {"student": student})
```

templates/list.html

```
<h2>Students</h2>
{% for s in students %}
  <p>
   {{ s.name }}
   <a href="{% url 'student_detail' s.id %}">View</a>
  </p>
{% endfor %}
```

templates/detail.html

```
<h2>Student Detail</h2>
<p>Name: {{ student.name }}</p>
<p>Age: {{ student.age }}</p>
<p>Email: {{ student.email }}</p>
<a href="{% url 'student_list' %}">Back</a>
```

Output

127.0.0.1:8000/students/

Student List

Name: Amala | View

127.0.0.1:8000/students/1/

Student Detail

Name: Amala

Age: 21

Email: amala@gmail.com

Back

Program No: 27	Date: 12/12/2025

Program Title: Create a Django login and logout system Create a Django login and logout system. Implement dark mode and light mode toggle using HTML and CSS.

```
<!—Login-logout system and toggle dark and light mode
@Ann Jo Mathew
Roll No: 12
19/12/2025 -->
```

app/views.py

```
from django.shortcuts import render, redirect
from django.contrib.auth import authenticate, login, logout
from django.contrib.auth.decorators import login_required
def login_view(request):
    error = ""
    if request.method == "POST":
        username = request.POST.get("username")
        password = request.POST.get("password")
        user = authenticate(request, username=username, password=password)
        if user is not None:
            login(request, user)
            return redirect("dashboard")
        else:
            error = "Invalid username or password"
    return render(request, "login.html", {"error": error})
def logout_view(request):
    logout(request)
    return redirect("login")
@login_required
def dashboard(request):
    return render(request, "dashboard.html")
```

app/urls.py

```
from django.urls import path
from . import views
urlpatterns = [
    path("", views.dashboard, name="dashboard"),
    path("login/", views.login_view, name="login"),
    path("logout/", views.logout_view, name="logout"),
]
```

templates/dashboard.html

```
<!DOCTYPE html>
```

```
<html>
<head>
  <title>Dashboard</title>
  <style>
    body{ font-family: Arial, sans-serif; padding: 20px; }
  </style>
</head>
<body>
  <h2>Dashboard</h2>
  <p>You are logged in as: <b>{{ request.user.username }}</b></p>
  <a href="{% url 'logout' %}">Logout</a>
</body>
</html>
```

templates/login.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Login</title>
  <style>
    :root{
      --bg:#0f172a;
      --card:#111827;
      --text:#e5e7eb;
      --muted:#9ca3af;
      --border:#2b3446;
      --btn:#22c55e;
    }
    body.light{
      --bg:#f3f4f6;
      --card:#ffffff;
      --text:#111827;
      --muted:#6b7280;
      --border:#d1d5db;
      --btn:#2563eb;
    }
    body{
      margin:0;
      font-family: Arial, sans-serif;
      background: var(--bg);
      color: var(--text);
      min-height:100vh;
      display:flex;
      align-items:center;
      justify-content:center;
      padding:20px;
```

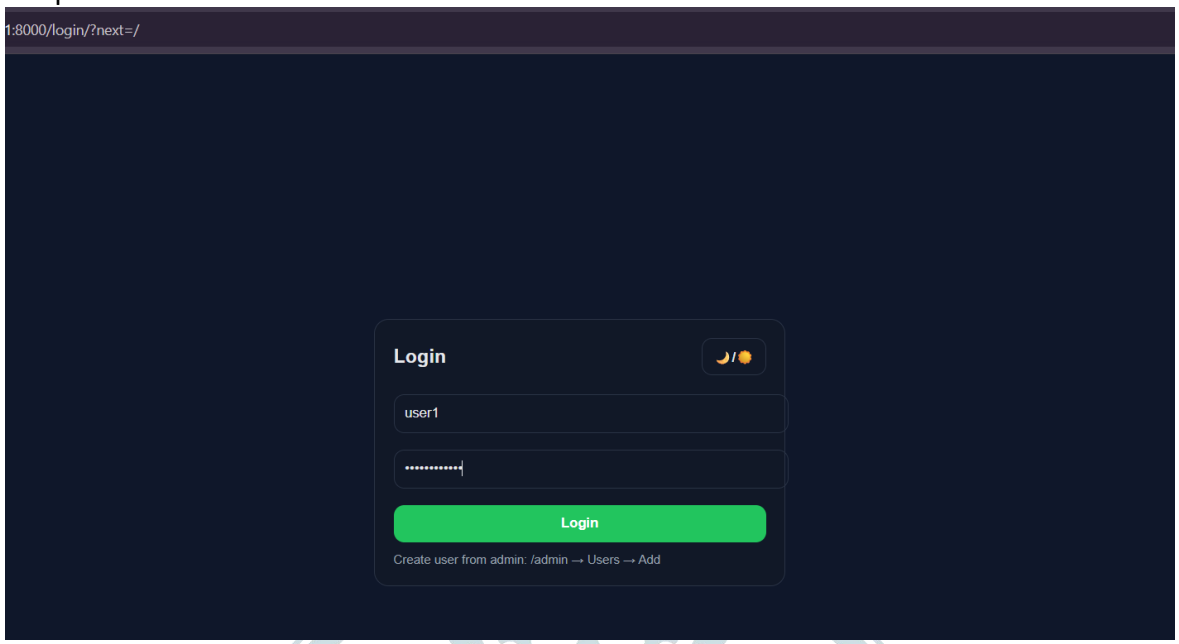
```
}
.card{
  width: 360px;
  background: var(--card);
  border: 1px solid var(--border);
  border-radius: 14px;
  padding: 18px;
}
.top{
  display:flex;
  align-items:center;
  justify-content:space-between;
  margin-bottom:10px;
}
input{
  width:100%;
  padding:10px;
  border-radius:10px;
  border:1px solid var(--border);
  background: transparent;
  color: var(--text);
  margin: 8px 0;
  outline:none;
}
button{
  width:100%;
  padding:10px;
  border:none;
  border-radius:10px;
  background: var(--btn);
  color: white;
  font-weight:700;
  cursor:pointer;
  margin-top: 8px;
}
.toggle{
  border:1px solid var(--border);
  background: transparent;
  color: var(--text);
  width:auto;
  padding:8px 10px;
  border-radius:10px;
  cursor:pointer;
  font-weight:600;
  margin-top:0;
}
.error{
```

```
    color:#ef4444;
    font-size: 13px;
    margin-top: 6px;
  }
  .muted{
    color: var(--muted);
    font-size: 12px;
    margin-top: 10px;
  }</style>
</head>
<body>
 <div class="card">
  <div class="top">
    <h3 style="margin:0;">Login</h3>
    <button class="toggle" type="button" id="themeBtn">🌙/☀</button>
  </div>
  <form method="POST">
    {% csrf_token %}
    <input type="text" name="username" placeholder="Username" required>
    <input type="password" name="password" placeholder="Password" required>
    <button type="submit">Login</button>
    {% if error %}
      <div class="error">{{ error }}</div>
    {% endif %}
  </form>
  <div class="muted">
    Create user from admin: /admin → Users → Add
  </div>
 </div>
 <script>
  // theme saved in localStorage
  const saved = localStorage.getItem("theme");
  if (saved === "light") document.body.classList.add("light");
  document.getElementById("themeBtn").addEventListener("click", () => {
    document.body.classList.toggle("light");
    localStorage.setItem("theme", document.body.classList.contains("light") ? "light" :
"dark");
  });
 </script>
</body>
</html>
```

pro/settings.py

```
LOGIN_URL = "login"
LOGIN_REDIRECT_URL = "dashboard"
LOGOUT_REDIRECT_URL = "login"
```

Output



Dashboard

You are logged in as: **user1**

Logout

Program No: 28	Date: 19/12/2025

Program Title:

```
<!-- Real-time digital clock using JavaScript
@Ann Jo Mathew
Roll No: 12
19/12/2025 -->
```

app/views.py

```
from django.shortcuts import render, redirect
from django.contrib.auth import authenticate, login, logout
from django.contrib.auth.decorators import login_required
def login_view(request):
    error = ""
    if request.method == "POST":
        username = request.POST.get("username")
        password = request.POST.get("password")
        user = authenticate(request, username=username, password=password)
        if user is not None:
            login(request, user)
            return redirect("dashboard")
        else:
            error = "Invalid username or password"
    return render(request, "login.html", {"error": error})
def logout_view(request):
    logout(request)
    return redirect("login")

@login_required
def dashboard(request):
    return render(request, "dashboard.html")
```

templates/dashboard.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Dashboard</title>
  <style>
    body{ font-family: Arial, sans-serif; padding: 20px; }
  </style>
</head>
<body>
  <h2>Dashboard</h2>
```

```
  <p>You are logged in as: <b>{{ request.user.username }}</b></p>
  <a href="{% url 'logout' %}">Logout</a>
</body>
</html>
```

pro/settings.py

```
LOGIN_URL = "login"
LOGIN_REDIRECT_URL = "dashboard"
LOGOUT_REDIRECT_URL = "login"
```

Output

Program No: 29	Date: 19/12/2025

Program Title: Create a Python Library

- Name your library appropriately (e.g., ai_helpers, text_utils, etc.).
- The library should include functions or classes that help in your AI task. Example functions:
  - get_response(prompt)
  - summarize_text(text)
  - format_response(text)
- Upload the library to PyPI (or create a local installable package).
- Include proper docstrings and usage examples.
- Write unit tests for the library functions (optional but recommended).
- Install and Use Your Library
- Install the library using pip install your-library-name.
- Import it into a Flask application.

Flask Web Application

- Homepage should include:
  - Textarea for user input
  - Submit button
  - Section to display AI responses

Optional Enhancements

- Style using Bootstrap or Material Design
- Maintain query history
- Handle errors gracefully

```
<!-- Dropdown-based content rendering using JavaScript
@Ann Jo Mathew
Roll No: 12
21/12/2025 -->
```

PyPi Package:summrai

__init__.py

```
from .client import AIClient
from .utils import summarize_text, format_response
__all__ = ["AIClient", "summarize_text", "format_response"]
__version__ = "0.1.2"
```

Client.py

```
import requests
class AIClient:
    def __init__(self, api_key: str, model: str = "gemini-2.0-flash"):
        if not api_key:
            raise ValueError("API key required. Pass api_key='YOUR_KEY'.")
```

```
        self.api_key = api_key
        self.model = model
        self.url =
"https://generativelanguage.googleapis.com/v1beta/openai/chat/completions"
    def get_response(self, prompt: str, max_tokens: int = 300) -> str:
        headers = {
            "Content-Type": "application/json",
            "Authorization": f"Bearer {self.api_key}"
        }
        body = {
            "model": self.model,
            "messages": [{"role": "user", "content": prompt}],
            "max_tokens": max_tokens
        }
        response = requests.post(self.url, json=body, headers=headers)
        if response.status_code != 200:
            raise Exception(f"API request failed: {response.text}")
        data = response.json()
        return data["choices"][0]["message"]["content"].strip()
```

Utils.py

```
def summarize_text(text: str, client) -> str:
    prompt = f"Summarize this:\n{text}"
    return client.get_response(prompt)
def format_response(text: str) -> str:
    return text.strip()
```

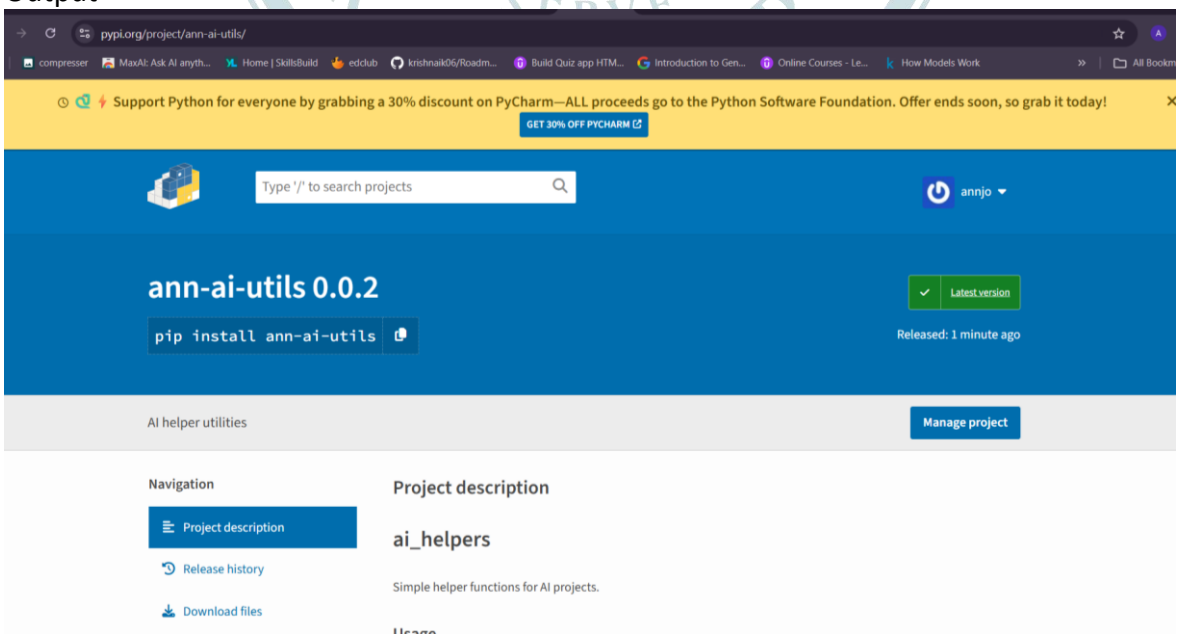
Flask ai assistant app

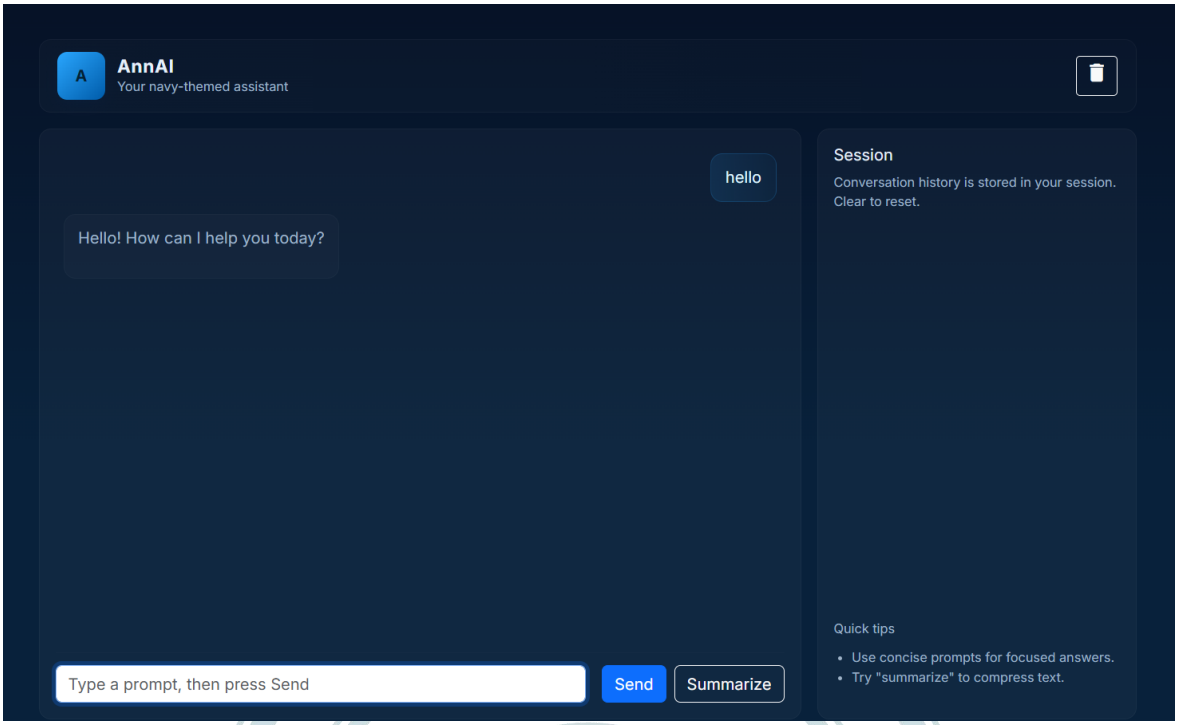
app.py

```
from flask import Flask, render_template, request, session, redirect, url_for
from ai_web_helper import get_response, summarize_text, format_response
import os
from dotenv import load_dotenv
load_dotenv()
app = Flask(__name__)
app.secret_key = os.environ.get("SECRET_KEY", "super_secret_key_for_dev") # Needed
for session
API_KEY = os.environ.get("GEMINI_API_KEY", "")
@app.route('/', methods=['GET', 'POST'])
def index():
    if 'history' not in session:
        session['history'] = []
    if request.method == 'POST':
```

```
    user_input = request.form.get('user_input')
    action = request.form.get('action')
    if user_input:
        if not API_KEY:
            response_text = "Error: API Key is missing. Please set GEMINI_API_KEY
environment variable."
            session['history'].append({'role': 'user', 'content': user_input})
            session['history'].append({'role': 'ai', 'content': response_text})
        else:
            session['history'].append({'role': 'user', 'content': user_input})
            if action == 'summarize':
                raw_response = summarize_text(user_input, API_KEY)
            else:
                raw_response = get_response(user_input, API_KEY)
            response_text = format_response(raw_response)
            session['history'].append({'role': 'ai', 'content': response_text})
        session.modified = True
        return redirect(url_for('index'))
    return render_template('index.html', history=session.get('history', []))


@app.route('/clear', methods=['POST'])
def clear_history():
    session.pop('history', None)
    return redirect(url_for('index'))
if __name__ == '__main__':
    app.run(debug=True)
```

Output





LEARN • SERVE • EXCEL
RAJAGIRI

Program No: 30	Date: 19/12/2025

Program Title: Online Resume / Portfolio Website
Create a personal online resume or portfolio website using Flask and deploy it on any free platform.
The website must include:

- About Me section
- Education & Skills
- Projects / Achievements
- ⍰ Contact Information

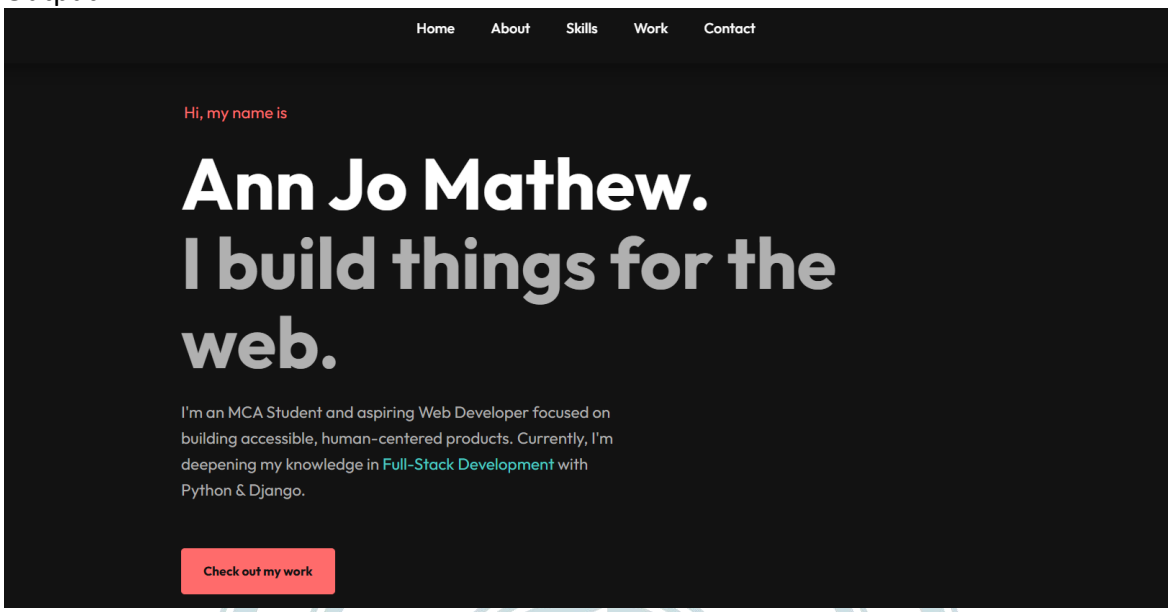
```
<!-- student details using JSON
@Ann Jo Mathew
Roll No: 12
21/12/2025 -->
```

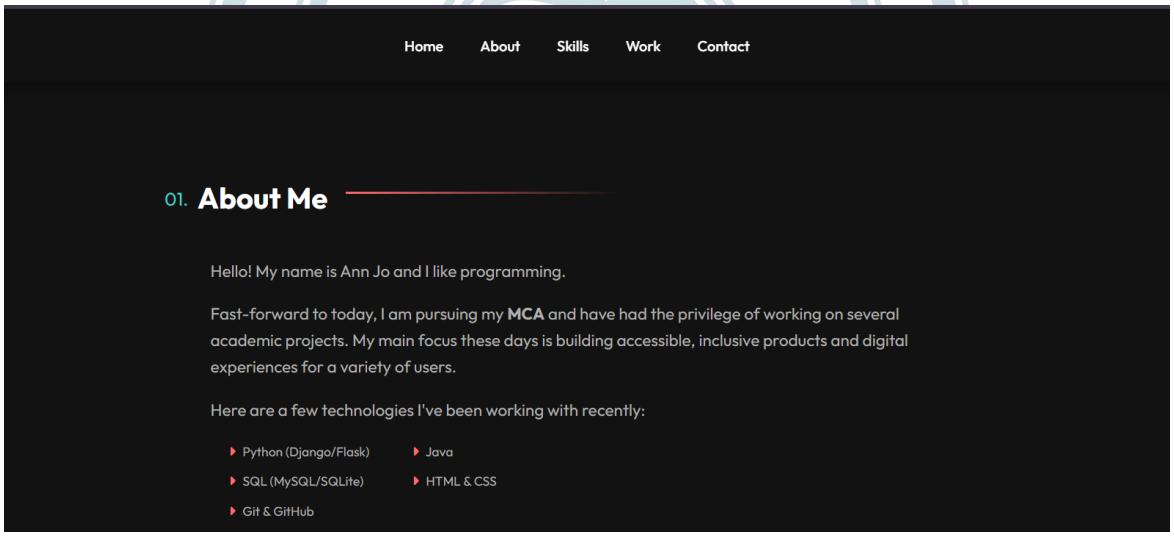
app.py

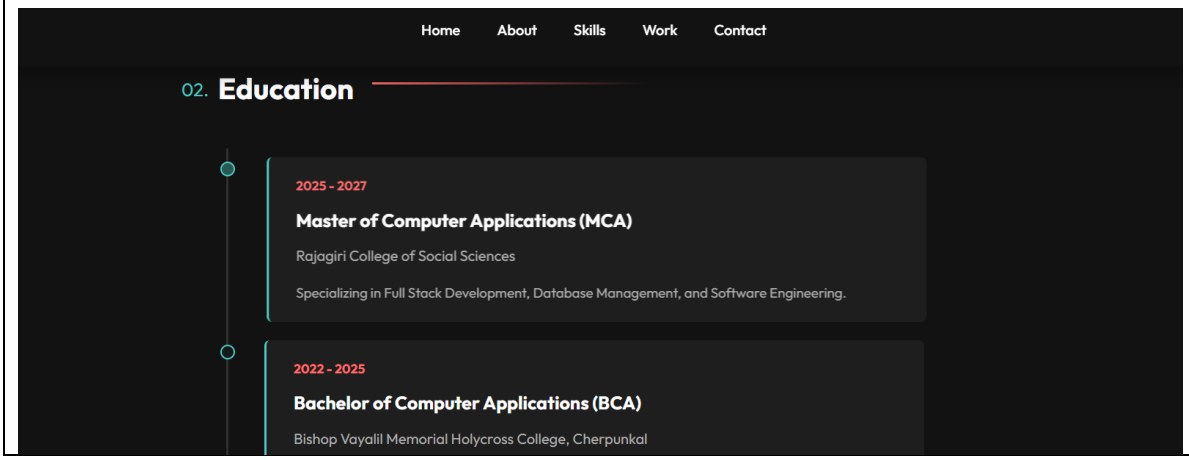
```
from flask import Flask, render_template
app = Flask(__name__)
projects = [
    {
        "title": "Recipe Management System",
        "description": "A web-based platform to add, browse, search, filter, rate, and
bookmark recipes. Built using Python, Django, and SQLite.",
        "github": "https://github.com/ann-jo-mathew/recipe_system/tree/ann",
        "tech": "Python, Django, SQLite",
        "image": "images/projects/Screenshot (79).png"
    },
]
education = [
    {
        "degree": "Master of Computer Applications (MCA)",
        "institution": "Rajagiri College of Social Sciences",
        "year": "2025 - 2027",
        "description": "Specializing in Full Stack Development, Database Management, and
Software Engineering."
    },
    {
        "degree": "Bachelor of Computer Applications (BCA)",
        "institution": "Bishop Vayalil Memorial Holycross College, Cherpunkal",
        "year": "2022 - 2025",
        "description": "Graduated with CGPA of 8.95. Built strong fundamentals in C, Java,
and Web Technologies."
    }
]
@app.route('/')
def home():
```

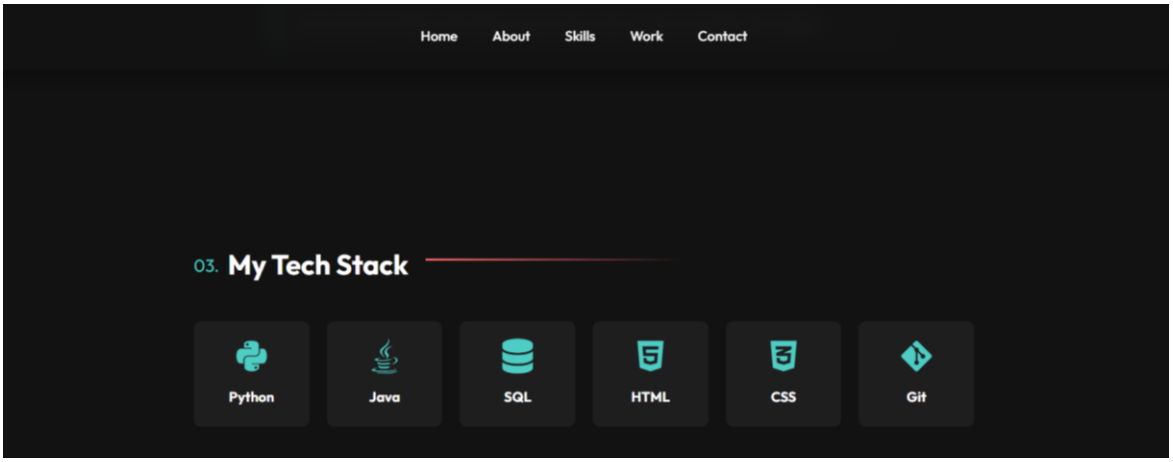
```
    return render_template('index.html', projects=projects, education=education)
if __name__ == '__main__':
    app.run(debug=True)
```

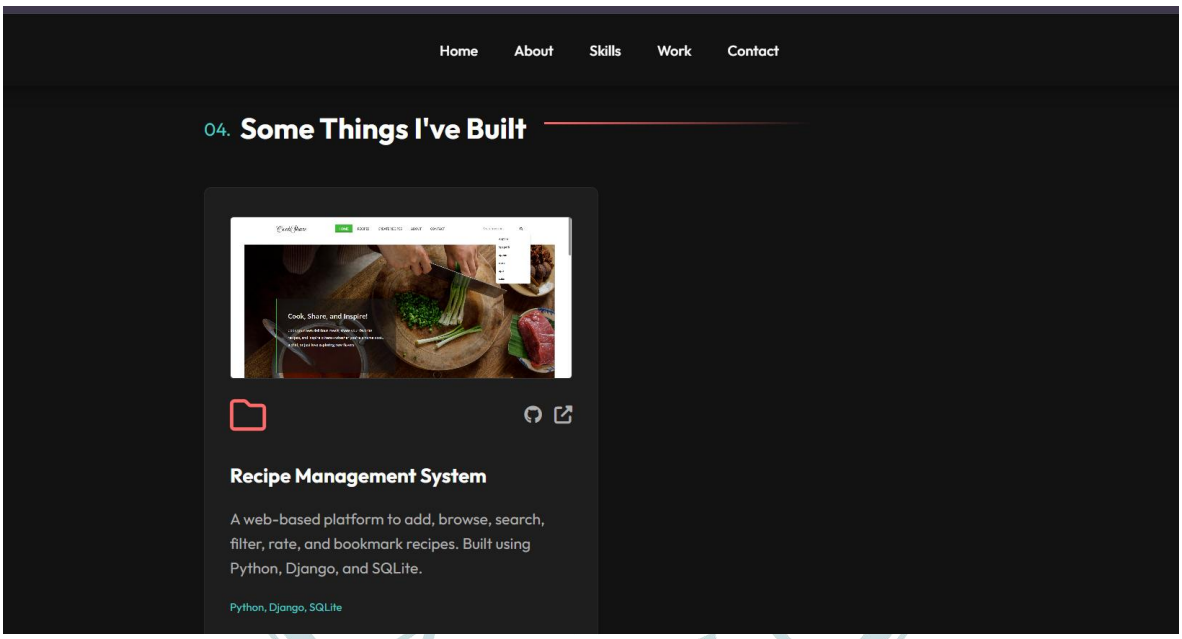
Output

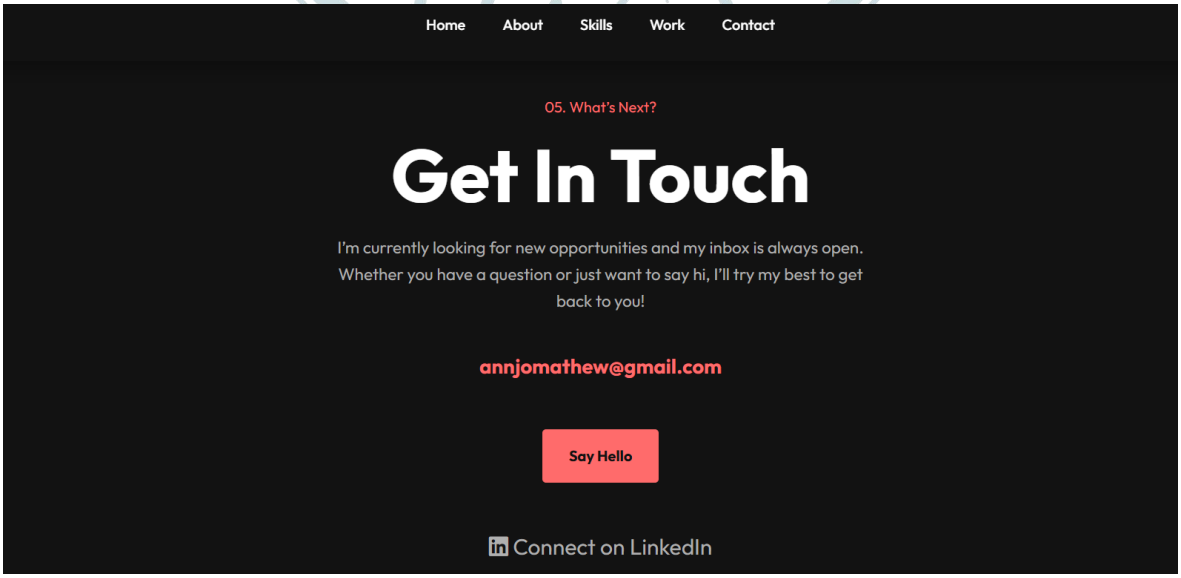







Home
About
Skills
Work
Contact
03. My Tech Stack
Python
Java
SQL
HTML
CSS
Git


Home
About
Skills
Work
Contact
04. Some Things I've Built
Cook, Share, and Inspire!
Recipe Management System
A web-based platform to add, browse, search, filter, rate, and bookmark recipes. Built using Python, Django, and SQLite.
Python, Django, SQLite


Home
About
Skills
Work
Contact
05. What's Next?
Get In Touch
I'm currently looking for new opportunities and my inbox is always open. Whether you have a question or just want to say hi, I'll try my best to get back to you!
annjomathew@gmail.com
Say Hello
Connect on LinkedIn

Program No: 31	Date: 12/12/2025

Program Title: Create a role-based authentication system using Django.

```
<!—role-based authetication
@Ann Jo Mathew
Roll No: 12
19/12/2025 -->
```

models.py

```
from django.db import models
from django.contrib.auth.models import User
class Profile(models.Model):
    ROLE_CHOICES = [
        ('ADMIN', 'ADMIN'),
        ('STAFF', 'STAFF'),
        ('STUDENT', 'STUDENT'),
    ]
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    role = models.CharField(max_length=20, choices=ROLE_CHOICES, default='STUDENT')
    def __str__(self):
        return self.user.username
```

urls.py

```
from django.urls import path
from . import views
urlpatterns = [
    path('', views.user_login, name='login'),
    path('login/', views.user_login, name='login_page'),
    path('register/', views.register, name='register'),
    path('logout/', views.user_logout, name='logout'),
    path('dashboard/', views.dashboard, name='dashboard'),
    path('admin_dashboard/', views.admin_dashboard, name='admin_dashboard'),
    path('staff_dashboard/', views.staff_dashboard, name='staff_dashboard'),
    path('student_dashboard/', views.student_dashboard, name='student_dashboard'),
]
```

views.py

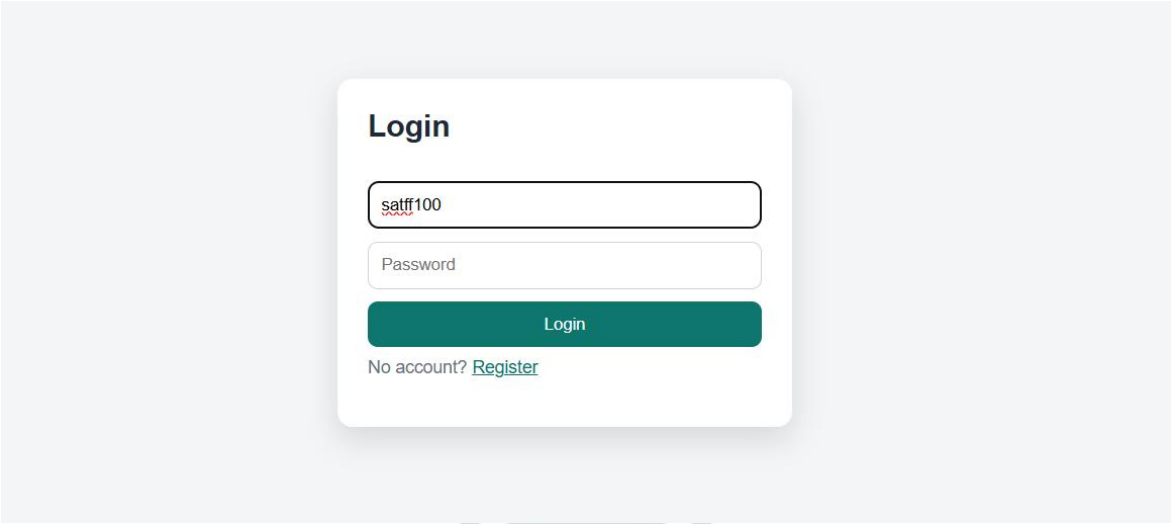
```
from django.shortcuts import render, redirect
from django.contrib.auth.models import User
from django.contrib.auth import authenticate, login, logout
from django.contrib.auth.decorators import login_required

from .models import Profile

# Registration with role
def register(request):
    if request.method == 'POST':
        username = request.POST.get('username')
        password = request.POST.get('password')
        role = request.POST.get('role')
        if User.objects.filter(username=username).exists():
            return render(request, 'register.html', {'error': 'Username already exists'})
        user = User.objects.create_user(username=username, password=password)
        Profile.objects.create(user=user, role=role)
        return redirect('login')
    return render(request, 'register.html')

# Login
def user_login(request):
    if request.user.is_authenticated:
        profile, _ = Profile.objects.get_or_create(user=request.user, defaults={'role':
'STUDENT'})
        if profile.role == 'ADMIN':
            return redirect('admin_dashboard')
        if profile.role == 'STAFF':
            return redirect('staff_dashboard')
        return redirect('student_dashboard')
    if request.method == 'POST':
        username = request.POST.get('username')
        password = request.POST.get('password')
        user = authenticate(request, username=username, password=password)
        if user:
            login(request, user)
            profile, _ = Profile.objects.get_or_create(user=user, defaults={'role': 'STUDENT'})
            if profile.role == 'ADMIN':
                return redirect('admin_dashboard')
            if profile.role == 'STAFF':
                return redirect('staff_dashboard')
            return redirect('student_dashboard')
        return render(request, 'login.html', {'error': 'Invalid username or password'})
    return render(request, 'login.html')
```

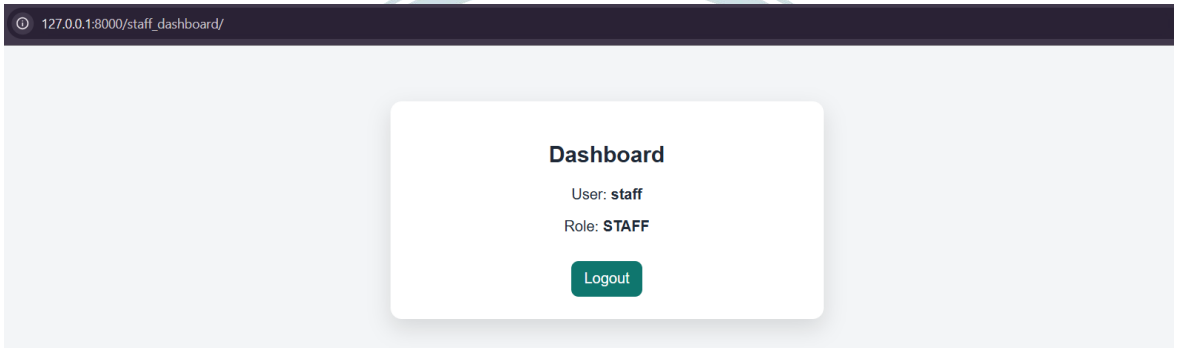
```
def user_logout(request):
    logout(request)
    return redirect('login')

@login_required
def dashboard(request):
    profile, _ = Profile.objects.get_or_create(user=request.user, defaults={'role':
'STUDENT'})
    return render(request, 'dashboard.html', {'role': profile.role})

@login_required
def admin_dashboard(request):
    profile, _ = Profile.objects.get_or_create(user=request.user, defaults={'role':
'STUDENT'})
    if profile.role != 'ADMIN':
        return redirect('dashboard')
    return render(request, 'dashboard.html', {'role': profile.role})

@login_required
def staff_dashboard(request):
    profile, _ = Profile.objects.get_or_create(user=request.user, defaults={'role':
'STUDENT'})
    if profile.role != 'STAFF':
        return redirect('dashboard')
    return render(request, 'dashboard.html', {'role': profile.role})

@login_required
def student_dashboard(request):
    profile, _ = Profile.objects.get_or_create(user=request.user, defaults={'role':
'STUDENT'})
    if profile.role != 'STUDENT':
        return redirect('dashboard')
    return render(request, 'dashboard.html', {'role': profile.role})
```

admin.py

```
from django.contrib import admin
from .models import Profile

admin.site.register(Profile)
```

Output


Login
satff100
Password
Login
No account? Register


127.0.0.1:8000/staff_dashboard/
Dashboard
User: staff
Role: STAFF
Logout

Program No: 32	Date: 19/12/2025

Program Title: Develop a student management system with CRUD operations.

```
<!—Student Management System
@Ann Jo Mathew
Roll No: 12
19/12/2025 -->
```

views.py

```
from django.shortcuts import render, redirect, get_object_or_404
from django.contrib.auth.decorators import login_required
from .models import Student

@login_required
def student_list(request):
    students = Student.objects.all()
    return render(request, 'app/list.html', {'students': students})

@login_required
def student_create(request):
    if request.method == "POST":
        name = request.POST.get('name')
        age = request.POST.get('age')
        email = request.POST.get('email')
        department = request.POST.get('department')
        Student.objects.create(name=name, age=age, email=email,
department=department)
        return redirect('student_list')
    return render(request, 'app/form.html')

@login_required
def student_update(request, id):
    student = get_object_or_404(Student, id=id)
    if request.method == "POST":
        student.name = request.POST.get('name')
        student.age = request.POST.get('age')
        student.email = request.POST.get('email')
        student.department = request.POST.get('department')
        student.save()
        return redirect('student_list')
    return render(request, 'app/form.html', {'student': student})

@login_required
def student_delete(request, id):
```

```
    student = get_object_or_404(Student, id=id)
    if request.method == "POST":
        student.delete()
        return redirect('student_list')
    return render(request, 'app/confirm_delete.html', {'student': student})
```

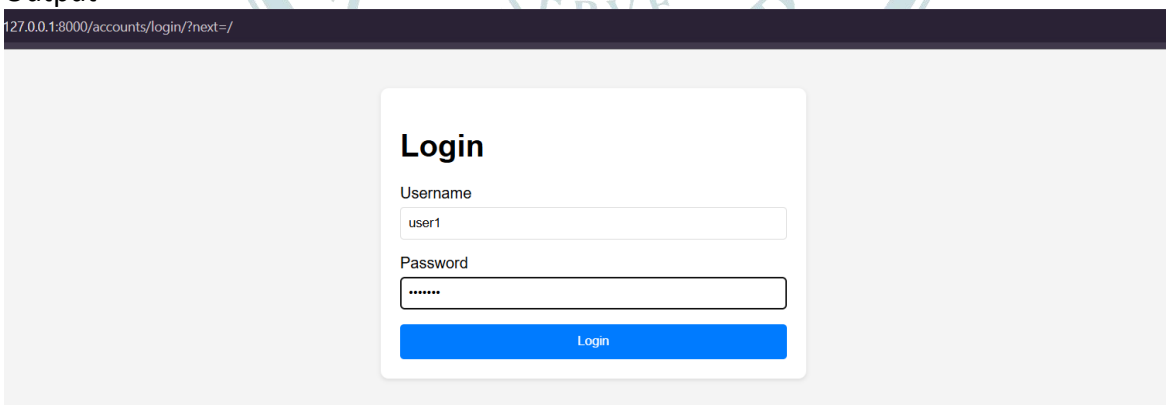
urls.py

```
from django.urls import path
from . import views
urlpatterns = [
    path('', views.student_list, name='student_list'),
    path('students/', views.student_list, name='student_list'),
    path('students/create/', views.student_create, name='student_create'),
    path('students/edit/<int:id>/', views.student_update, name='student_update'),
    path('students/delete/<int:id>/', views.student_delete, name='student_delete'),
]
```

models.py

```
from django.db import models
class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()
    email = models.EmailField()
    department = models.CharField(max_length=100)
    def __str__(self):
        return self.name
```

Output



127.0.0.1:8000

Student Management

Welcome, user1 | Logout

Add Student

Name	Age	Email	Department	Actions
Alice Johnson	20	alice@example.com	Computer Science	Edit Delete
Bob Smith	22	bob@example.com	Mathematics	Edit Delete
Charlie Brown	21	charlie@example.com	Physics	Edit Delete
Diana Prince	23	diana@example.com	Engineering	Edit Delete
Ethan Hunt	19	ethan@example.com	Biology	Edit Delete

127.0.0.1:8000/students/edit/1/

Edit Student

Name

Alice Johnson

Age

20

Email

alice@example.com

Department

Computer Science

Save Cancel

127.0.0.1:8000/students/delete/1/

Confirm Delete

Are you sure you want to delete student **Alice Johnson**?

Yes, Delete Cancel

Program No: 33	Date: 19/12/2025

Program Title: Create simple online feedback system with database integration.

```
<!—Online feedback System
@Ann Jo Mathew
Roll No: 12
21/12/2025 -->
```

models.py

```
from django.db import models
class Feedback(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField()
    rating = models.IntegerField()
    message = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)
    def __str__(self):
        return f"{self.name} - {self.rating} stars"
```

urls.py

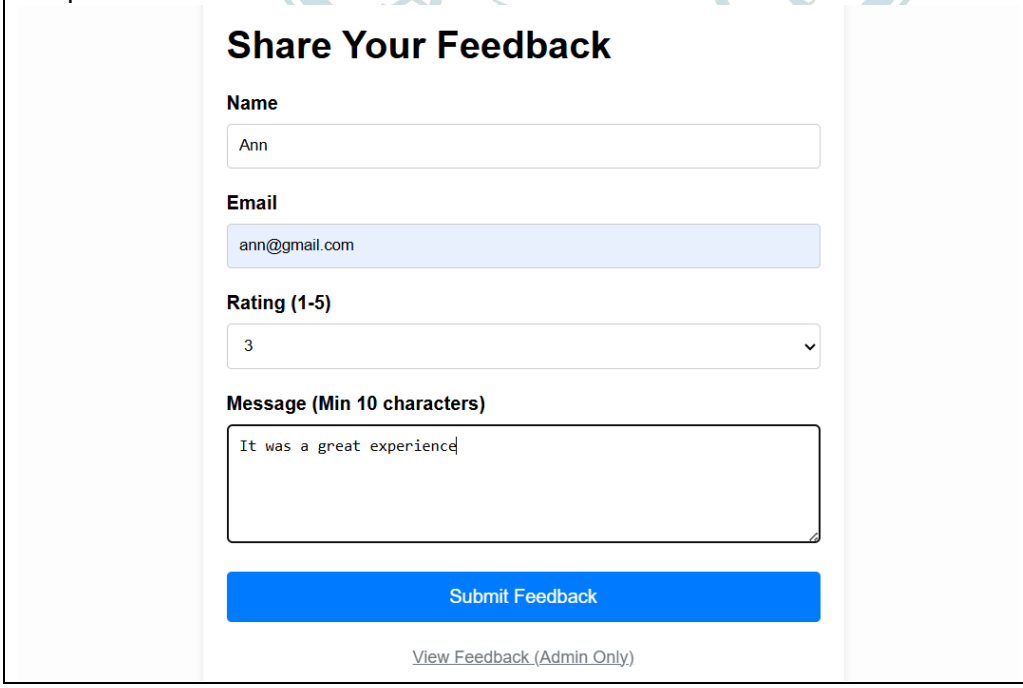
```
from django.urls import path
from . import views
urlpatterns = [
    path('', views.feedback_form, name='feedback_form'),
    path('list/', views.feedback_list, name='feedback_list'),
]
```

views.py

```
from django.shortcuts import render, redirect
from django.contrib.auth.decorators import login_required, user_passes_test
from .models import Feedback
def feedback_form(request):
    errors = {}
    success = False
    data = {}
    if request.method == "POST":
        data['name'] = request.POST.get('name', '').strip()
        data['email'] = request.POST.get('email', '').strip()
        data['rating'] = request.POST.get('rating', '')
        data['message'] = request.POST.get('message', '').strip()

        # Validation
```

```
        if not data['name']:
            errors['name'] = "Name is required."
        if not data['email']:
            errors['email'] = "Email is required."
        try:
            rating = int(data['rating'])
            if rating < 1 or rating > 5:
                errors['rating'] = "Rating must be between 1 and 5."
        except (ValueError, TypeError):
            errors['rating'] = "A valid rating is required."
        if len(data['message']) < 10:
            errors['message'] = "Message must be at least 10 characters long."
        if not errors:
            Feedback.objects.create(
                name=data['name'],
                email=data['email'],
                rating=int(data['rating']),
                message=data['message']
            )
            success = True
            data = {} # Clear form on success
    return render(request, 'app/form.html', {'errors': errors, 'success': success, 'data':
data})

@login_required
@user_passes_test(lambda u: u.is_staff)
def feedback_list(request):
    feedbacks = Feedback.objects.all().order_by('-created_at')
    return render(request, 'app/list.html', {'feedbacks': feedbacks})
```

Output



All Feedback

Hi, admin | Logout

From	Rating	Message	Date
Ann ann@gmail.com	3 / 5	It was a great experience	2026-03-03 20:46
Mike Rossi mike@example.com	3 / 5	Average experience, nothing special but it works.	2026-03-03 20:33
Jane Smith jane@example.com	4 / 5	Pretty good, but there is room for improvement in response time.	2026-03-03 20:33
John Doe john@example.com	5 / 5	Excellent service and very helpful staff!	2026-03-03 20:33

Back to Form

Program No: 34	Date: 19/12/2025
Program Title: Create a blog application.	

```
<!—Blog application
@Ann Jo Mathew
Roll No: 12
21/12/2025 -->
```

models.py

```
from django.db import models
from django.contrib.auth.models import User

class Post(models.Model):
    title = models.CharField(max_length=200)
    content = models.TextField()
    author = models.ForeignKey(User, on_delete=models.CASCADE)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.title
```

urls.py

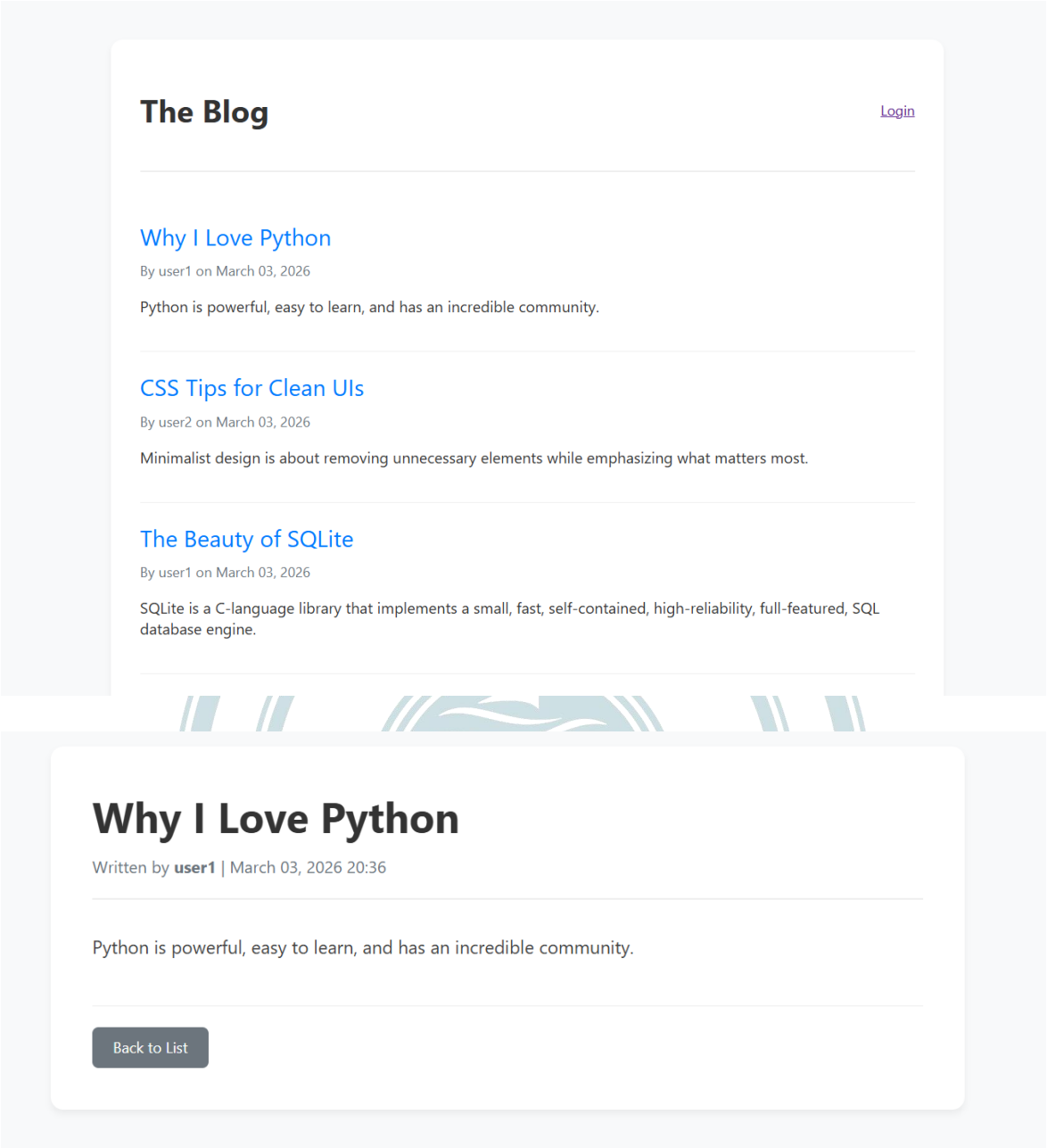
```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.post_list, name='post_list'),
    path('post/<int:id>/', views.post_detail, name='post_detail'),
    path('post/create/', views.post_create, name='post_create'),
    path('post/edit/<int:id>/', views.post_edit, name='post_edit'),
    path('post/delete/<int:id>/', views.post_delete, name='post_delete'),
]
```

views.py

```
from django.shortcuts import render, redirect, get_object_or_404
from django.contrib.auth.decorators import login_required
from django.http import HttpResponseForbidden
from .models import Post

def post_list(request):
    posts = Post.objects.all().order_by('-created_at')
    return render(request, 'app/list.html', {'posts': posts})
```

```
def post_detail(request, id):
    post = get_object_or_404(Post, id=id)
    return render(request, 'app/detail.html', {'post': post})

@login_required
def post_create(request):
    if request.method == "POST":
        title = request.POST.get('title')
        content = request.POST.get('content')
        Post.objects.create(title=title, content=content, author=request.user)
        return redirect('post_list')
    return render(request, 'app/form.html')

@login_required
def post_edit(request, id):
    post = get_object_or_404(Post, id=id)
    if not (request.user == post.author or request.user.is_staff):
        return HttpResponseForbidden("You are not allowed to edit this post.")

    if request.method == "POST":
        post.title = request.POST.get('title')
        post.content = request.POST.get('content')
        post.save()
        return redirect('post_detail', id=post.id)
    return render(request, 'app/form.html', {'post': post})

@login_required
def post_delete(request, id):
    post = get_object_or_404(Post, id=id)
    if not (request.user == post.author or request.user.is_staff):
        return HttpResponseForbidden("You are not allowed to delete this post.")

    if request.method == "POST":
        post.delete()
        return redirect('post_list')
    return render(request, 'app/confirm_delete.html', {'post': post})
```

Output


The Blog
Login
Why I Love Python
By user1 on March 03, 2026
Python is powerful, easy to learn, and has an incredible community.
CSS Tips for Clean UIs
By user2 on March 03, 2026
Minimalist design is about removing unnecessary elements while emphasizing what matters most.
The Beauty of SQLite
By user1 on March 03, 2026
SQLite is a C-language library that implements a small, fast, self-contained, high-reliability, full-featured, SQL database engine.
Why I Love Python
Written by user1 | March 03, 2026 20:36
Python is powerful, easy to learn, and has an incredible community.
Back to List

Program No: 35	Date: 12/12/2025

Program Title: Implement session management in Django.

```
<!—session management
@Ann Jo Mathew
Roll No: 12
19/12/2025 -->
```

models.py

```
from django.db import models

class Note(models.Model):
    title = models.CharField(max_length=100)
    message = models.TextField()

    def __str__(self):
        return self.title
```

urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.home, name='home'),
    path('set-name/', views.set_name, name='set_name'),
    path('set-theme/', views.set_theme, name='set_theme'),
    path('clear-session/', views.clear_session, name='clear_session'),
    path('login/', views.user_login, name='login'),
    path('logout/', views.user_logout, name='logout'),
]
```

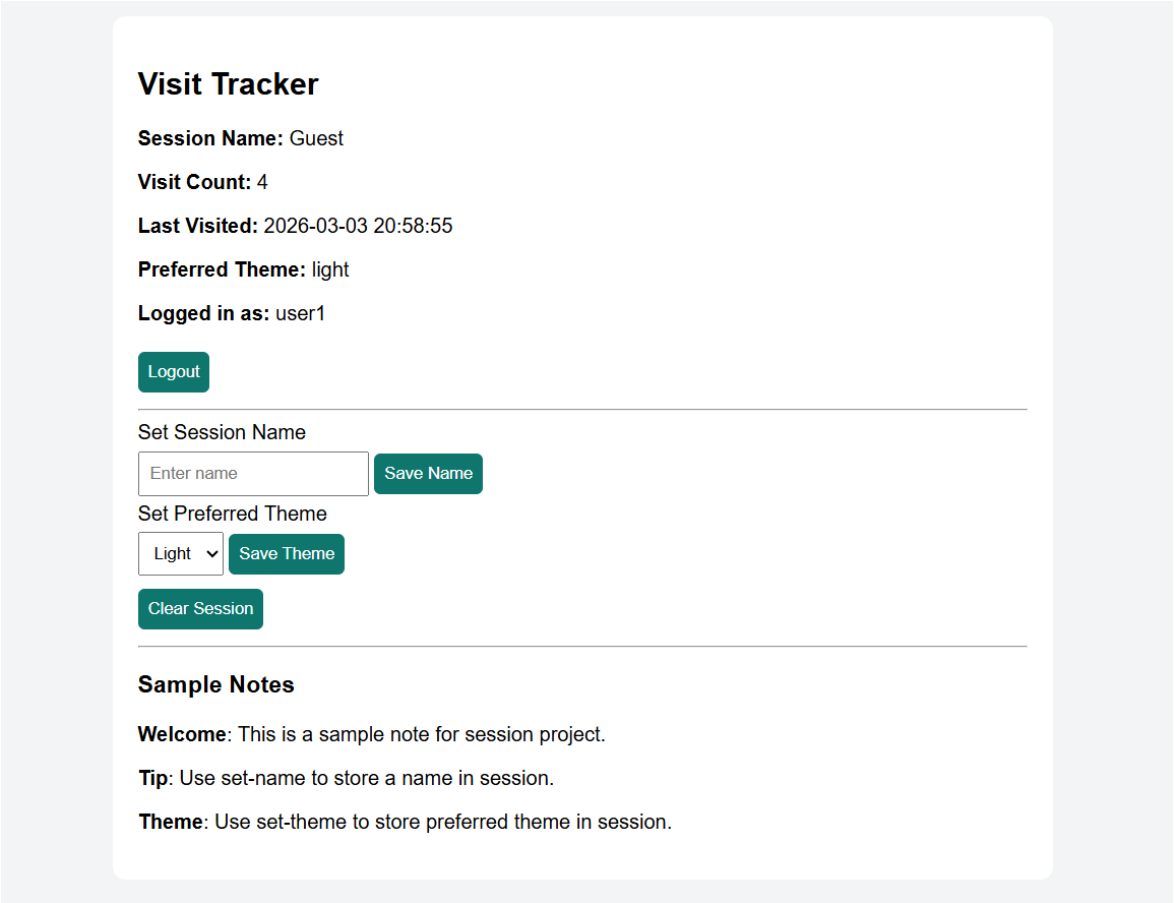
views.py

```
from django.contrib.auth import authenticate, login, logout
from django.shortcuts import redirect, render
from django.utils import timezone
from .models import Note

def home(request):
    count = request.session.get('visit_count', 0) + 1
    request.session['visit_count'] = count

    last_visited = request.session.get('last_visited')
```

```
    now = timezone.now().strftime('%Y-%m-%d %H:%M:%S')
    request.session['last_visited'] = now
    username = request.session.get('username', 'Guest')
    theme = request.session.get('theme', 'light')
    notes = Note.objects.all()
    context = {
        'visit_count': count,
        'last_visited': last_visited,
        'username': username,
        'theme': theme,
        'notes': notes,
    }
    return render(request, 'app/home.html', context)


def set_name(request):
    if request.method == 'POST':
        name = request.POST.get('username', '').strip()
        if name:
            request.session['username'] = name
    return redirect('home')


def set_theme(request):
    if request.method == 'POST':
        theme = request.POST.get('theme', 'light')
        if theme in ['light', 'dark']:
            request.session['theme'] = theme
    return redirect('home')


def clear_session(request):
    request.session.flush()
    return redirect('home')


def user_login(request):
    if request.method == 'POST':
        username = request.POST.get('username')
        password = request.POST.get('password')
        user = authenticate(request, username=username, password=password)

        if user is not None:
            login(request, user)
            return redirect('home')
        return render(request, 'app/login.html', {'error': 'Invalid username or password'})
    return render(request, 'app/login.html')
```

```
def user_logout(request):
    logout(request)
    return redirect('home')
```

Output



Program No: 36	Date: 19/12/2025

Program Title: Implement pagination and search in a Django application.

```
<!—Pagination and search
@Ann Jo Mathew
Roll No: 12
19/12/2025 -->
```

models.py

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()
    email = models.EmailField(unique=True)

    def __str__(self):
        return self.name
```

urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path('students/', views.student_list, name='student_list'),
]
```

views.py

```
from django.core.paginator import Paginator
from django.db.models import Q
from django.shortcuts import render
from .models import Student

def student_list(request):
    q = request.GET.get('q', '').strip()
    students = Student.objects.all().order_by('id')
    if q:
        students = students.filter(Q(name__icontains=q) | Q(email__icontains=q))
    paginator = Paginator(students, 5)
    page_number = request.GET.get('page')
    page_obj = paginator.get_page(page_number)
    return render(request, 'app/student_list.html', {'page_obj': page_obj, 'q': q})
```

Output

127.0.0.1:8000/students/?q=&page=2

Student List

Search by name or email | Search | Clear

Name	Age	Email
Student 6	19	student6@example.com
Student 7	20	student7@example.com
Student 8	21	student8@example.com
Student 9	22	student9@example.com
Student 10	18	student10@example.com

Previous Page 2 of 5 Next

127.0.0.1:8000/students/?q=student+7

Student List

student 7 | Search | Clear

Name	Age	Email
Student 7	20	student7@example.com

Page 1 of 1

Program No: 37	Date: 19/12/2025

Program Title: Implement form validation and error handling in Django.

```
<!—Form validation and error handling
@Ann Jo Mathew
Roll No: 12
21/12/2025 -->
```

models.py

```
from django.db import models
class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()
    email = models.EmailField(unique=True)
    def __str__(self):
        return self.name
```

urls.py

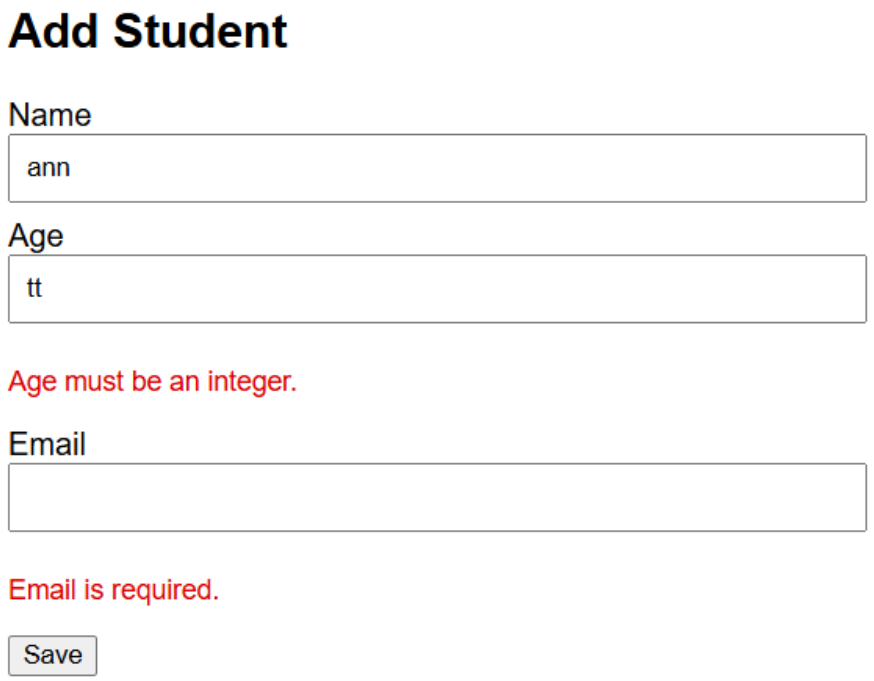
```
from django.urls import path
from . import views
urlpatterns = [
    path('student/add/', views.student_add, name='student_add'),
]
```

views.py

```
from django.core.validators import validate_email
from django.core.exceptions import ValidationError
from django.shortcuts import render
from .models import Student
def student_add(request):
    errors = {}
    data = {}
    success = ''
    if request.method == 'POST':
        name = request.POST.get('name', '').strip()
        age = request.POST.get('age', '').strip()
        email = request.POST.get('email', '').strip()
        data = {'name': name, 'age': age, 'email': email}
        if not name:
            errors['name'] = 'Name is required.'
        elif len(name) < 3:
            errors['name'] = 'Name must be at least 3 characters.'
```

```
        if not age:
            errors['age'] = 'Age is required.'
        else:
            try:
                age_int = int(age)
                if age_int < 1 or age_int > 120:
                    errors['age'] = 'Age must be between 1 and 120.'
            except ValueError:
                errors['age'] = 'Age must be an integer.'
        if not email:
            errors['email'] = 'Email is required.'
        else:
            try:
                validate_email(email)
            except ValidationError:
                errors['email'] = 'Enter a valid email address.'
            if Student.objects.filter(email=email).exists():
                errors['email'] = 'Email already exists.'
        if not errors:
            try:
                Student.objects.create(name=name, age=age_int, email=email)
                success = 'Student saved successfully.'
                data = {}
            except Exception:
                errors['general'] = 'Something went wrong while saving data.'
    return render(request, 'app/student_form.html', {'errors': errors, 'data': data,
'success': success})
```

Output



Program No: 38	Date: 19/12/2025
Program Title: Create a secure authentication system with password hashing.	

```
<!—Authentication system with hashing
@Ann Jo Mathew
Roll No: 12
21/12/2025 -->
```

models.py

```
from django.db import models

class LoginLog(models.Model):
    username = models.CharField(max_length=150)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.username
```

urls.py

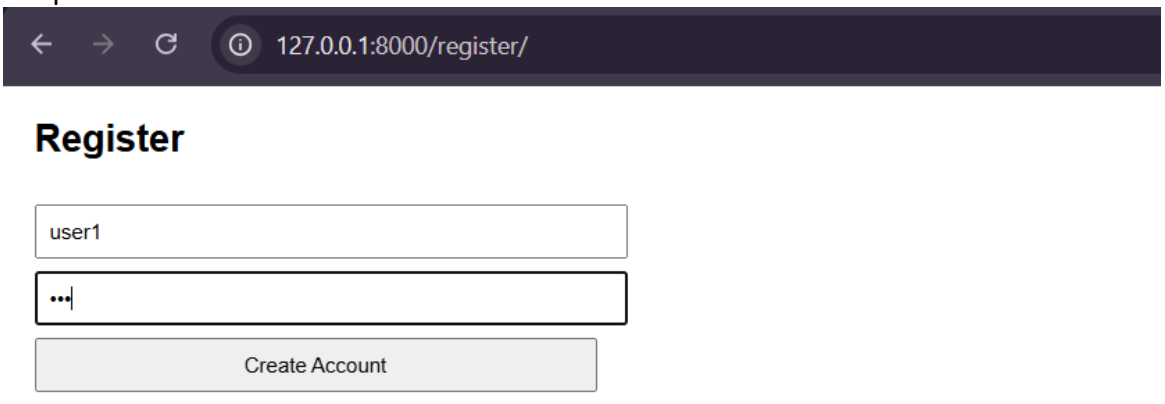
```
from django.urls import path
from . import views

urlpatterns = [
    path('register/', views.register_view, name='register'),
    path('login/', views.login_view, name='login'),
    path('logout/', views.logout_view, name='logout'),
    path('dashboard/', views.dashboard_view, name='dashboard'),
]
```

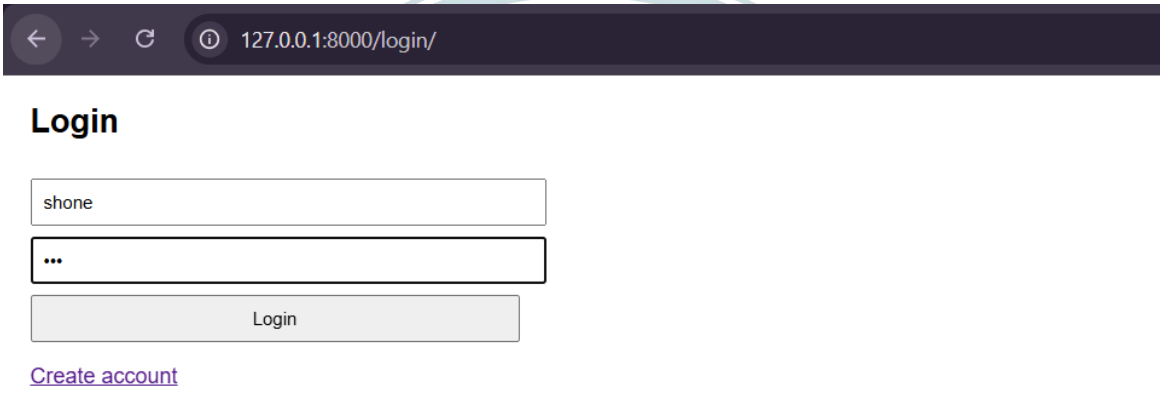
views.py

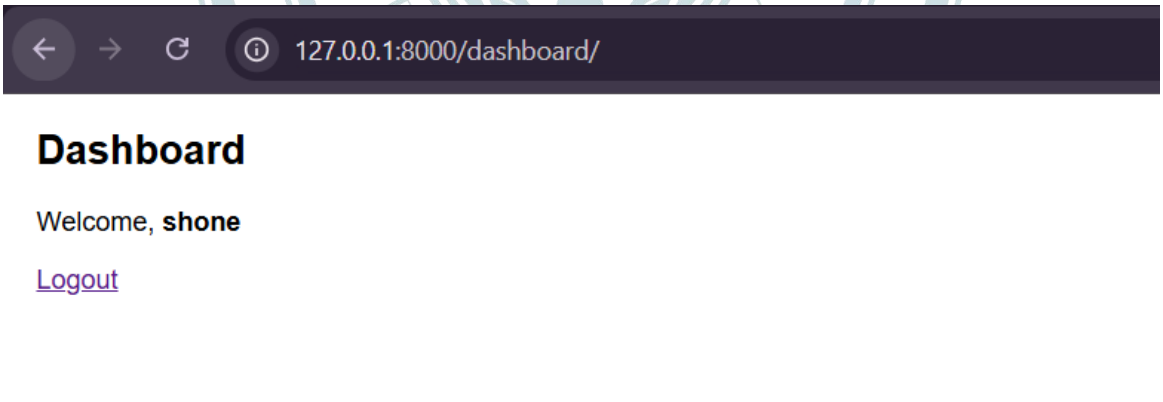
```
from django.contrib.auth import authenticate, login, logout
from django.contrib.auth.decorators import login_required
from django.contrib.auth.models import User
from django.shortcuts import redirect, render

from .models import LoginLog


def register_view(request):
    error = ''

    if request.method == 'POST':
        username = request.POST.get('username', '').strip()
```

```
        password = request.POST.get('password', '').strip()

        if not username or not password:
            error = 'Username and password are required.'
        elif User.objects.filter(username=username).exists():
            error = 'Username already exists.'
        else:
            User.objects.create_user(username=username, password=password)
            return redirect('login')

    return render(request, 'app/register.html', {'error': error})


def login_view(request):
    error = ''

    if request.method == 'POST':
        username = request.POST.get('username', '').strip()
        password = request.POST.get('password', '').strip()
        user = authenticate(request, username=username, password=password)

        if user is not None:
            login(request, user)
            LoginLog.objects.create(username=user.username)
            return redirect('dashboard')
        error = 'Invalid username or password.'

    return render(request, 'app/login.html', {'error': error})


def logout_view(request):
    logout(request)
    return redirect('login')


@login_required
def dashboard_view(request):
    return render(request, 'app/dashboard.html')
```

Output


127.0.0.1:8000/register/
Register
user1
Create Account
Go to login


127.0.0.1:8000/login/
Login
shone
Login
Create account


127.0.0.1:8000/dashboard/
Dashboard
Welcome, shone
Logout

Program No: 39	Date: 12/12/2025

Program Title: Deploy a full stack application on a free hosting platform.

```
<!—Deploy
@Ann Jo Mathew
Roll No: 12
19/12/2025 -->
```

models.py

```
from django.db import models

class ContactMessage(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField()
    message = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"{self.name} ({self.email})"
```

views.py

```
from django.shortcuts import render, redirect
from .models import ContactMessage

def home(request):
    success = False
    error = ""

    if request.method == "POST":
        name = request.POST.get("name", "").strip()
        email = request.POST.get("email", "").strip()
        message = request.POST.get("message", "").strip()

        if not name or not email or not message:
            error = "All fields are required."
        elif len(message) < 10:
            error = "Message must be at least 10 characters."
        else:
            ContactMessage.objects.create(name=name, email=email, message=message)
            return redirect("/?sent=1")

    if request.GET.get("sent") == "1":
        success = True
```

```
    return render(request, "webapp/home.html", {"success": success, "error": error})
```

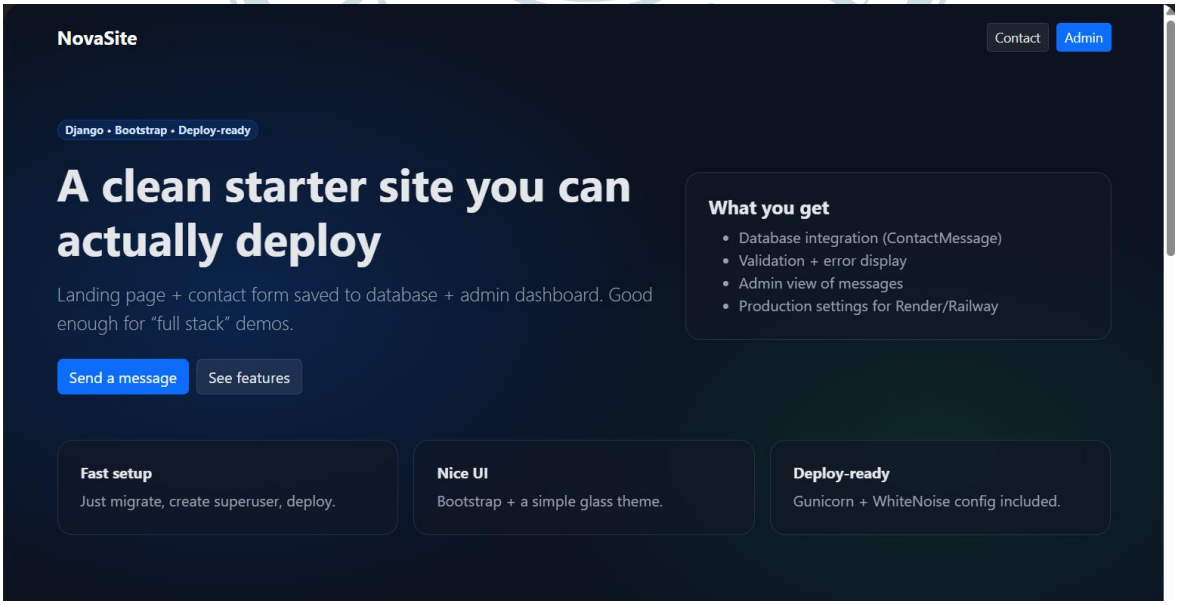
urls.py

```
from django.urls import path
from . import views

urlpatterns = [
path("", views.home, name="home"),
]
```

settings.py

```
import os
SECRET_KEY = os.environ.get("SECRET_KEY", "dev-only-change-me")
DEBUG = os.environ.get("DEBUG", "0") == "1"
ALLOWED_HOSTS = ["*"]

import dj_database_url

DATABASES = {
    "default": dj_database_url.config(default=f"sqlite:///{BASE_DIR/'db.sqlite3'}")
}
```

Output



Program No: 40	Date: 19/12/2025
Program Title: Develop a socially relevant mini project aligned with UN SDGs.	

```
<!—mini project
@Ann Jo Mathew
Roll No: 12
19/12/2025 -->
```

models.py

```
from django.db import models
from django.contrib.auth.models import User

class Note(models.Model):
    title = models.CharField(max_length=200)
    subject = models.CharField(max_length=100)
    semester = models.CharField(max_length=20)
    description = models.TextField(blank=True)
    file = models.FileField(upload_to='notes_files/')
    uploaded_by = models.ForeignKey(User, on_delete=models.CASCADE)
    uploaded_at = models.DateTimeField(auto_now_add=True)
    def __str__(self):
        return self.title
    class Meta:
        ordering = ['-uploaded_at']
```

urls.py

```
from django.urls import path
from . import views
urlpatterns = [
    path('', views.note_list, name='note_list'),
    path('upload/', views.note_upload, name='note_upload'),
    path('<int:pk>/', views.note_detail, name='note_detail'),
]
```

forms.py

```
from django import forms
from .models import Note

class NoteForm(forms.ModelForm):
    class Meta:
        model = Note
        fields = ['title', 'subject', 'semester', 'description', 'file']
```

```
        widgets = {
            'title': forms.TextInput(attrs={'class': 'form-control', 'placeholder': 'Note Title'}),
            'subject': forms.TextInput(attrs={'class': 'form-control', 'placeholder': 'e.g.
Mathematics'}),
            'semester': forms.TextInput(attrs={'class': 'form-control', 'placeholder': 'e.g.
Semester 3'}),
            'description': forms.Textarea(attrs={'class': 'form-control', 'rows': 4, 'placeholder':
'Optional description...'}),
            'file': forms.ClearableFileInput(attrs={'class': 'form-control'}),
        }
```

views.py

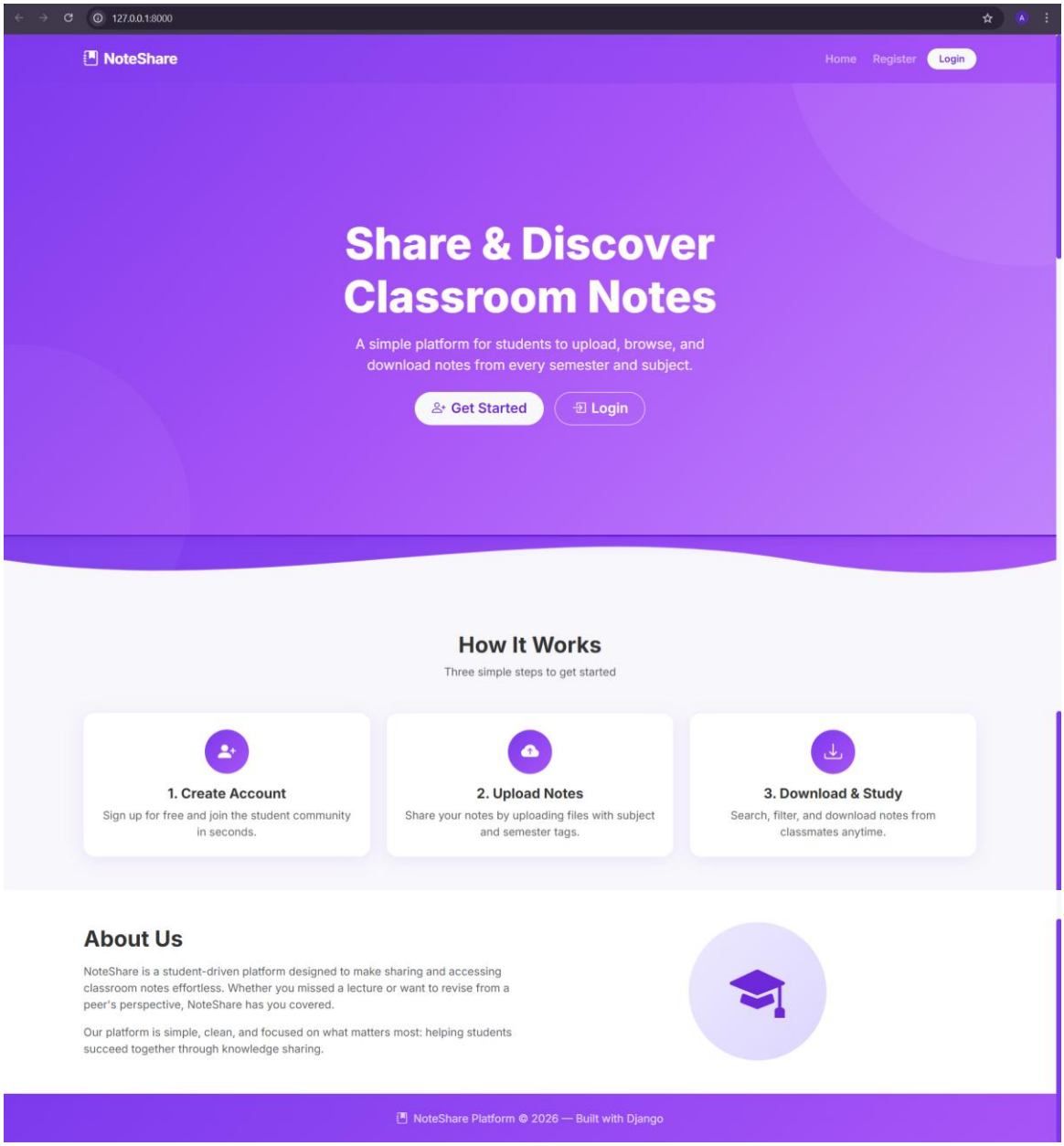
```
from django.shortcuts import render, redirect, get_object_or_404
from django.contrib.auth.decorators import login_required
from django.contrib import messages
from .models import Note
from .forms import NoteForm

@login_required
def note_list(request):
    """List all notes with search and filter support."""
    notes = Note.objects.all()

    # Get search/filter parameters
    query = request.GET.get('q', '').strip()
    subject = request.GET.get('subject', '').strip()
    semester = request.GET.get('semester', '').strip()

    # Apply filters
    if query:
        notes = notes.filter(title__icontains=query)
    if subject:
        notes = notes.filter(subject__icontains=subject)
    if semester:
        notes = notes.filter(semester__icontains=semester)

    context = {
        'notes': notes,
        'query': query,
        'subject': subject,
        'semester': semester,
    }
    return render(request, 'notes/note_list.html', context)

@login_required
def note_upload(request):
```

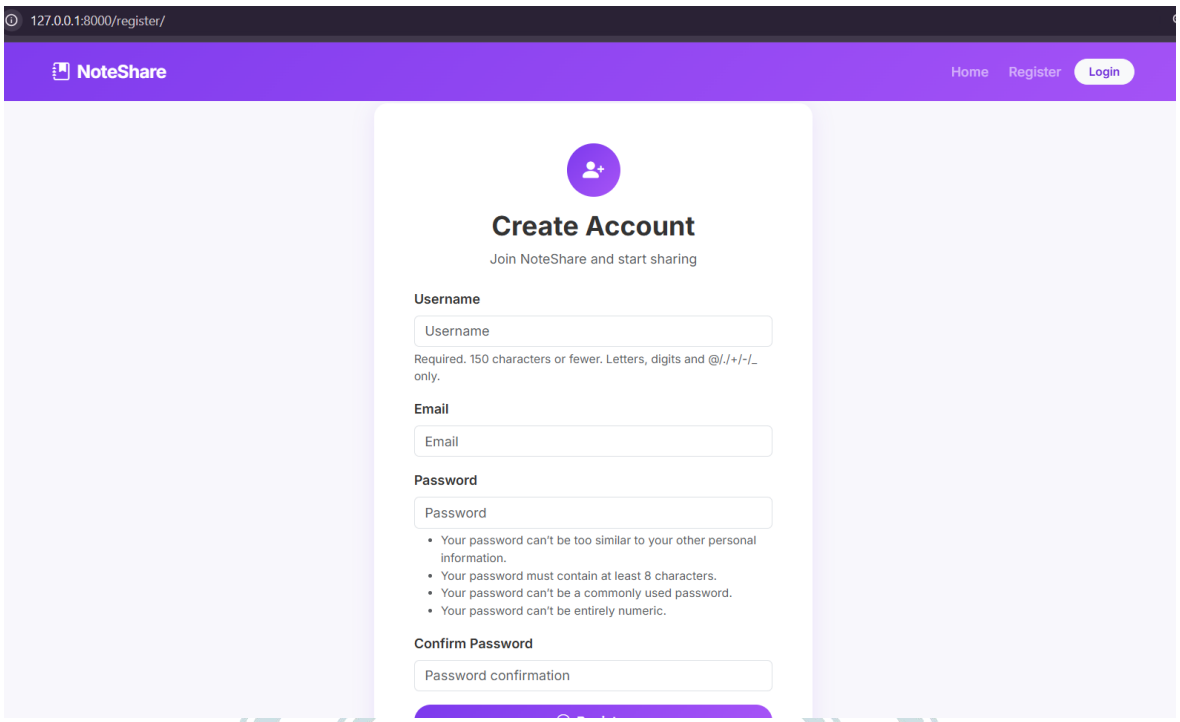
```
    """Handle note file upload."""
    if request.method == 'POST':
        form = NoteForm(request.POST, request.FILES)
        if form.is_valid():
            note = form.save(commit=False)
            note.uploaded_by = request.user
            note.save()
            messages.success(request, 'Note uploaded successfully!')
            return redirect('note_list')
        else:
            messages.error(request, 'Please correct the errors below.')
    else:
        form = NoteForm()

    return render(request, 'notes/note_upload.html', {'form': form})

@login_required
def note_detail(request, pk):
    """Display a single note with download link."""
    note = get_object_or_404(Note, pk=pk)
    return render(request, 'notes/note_detail.html', {'note': note})
```

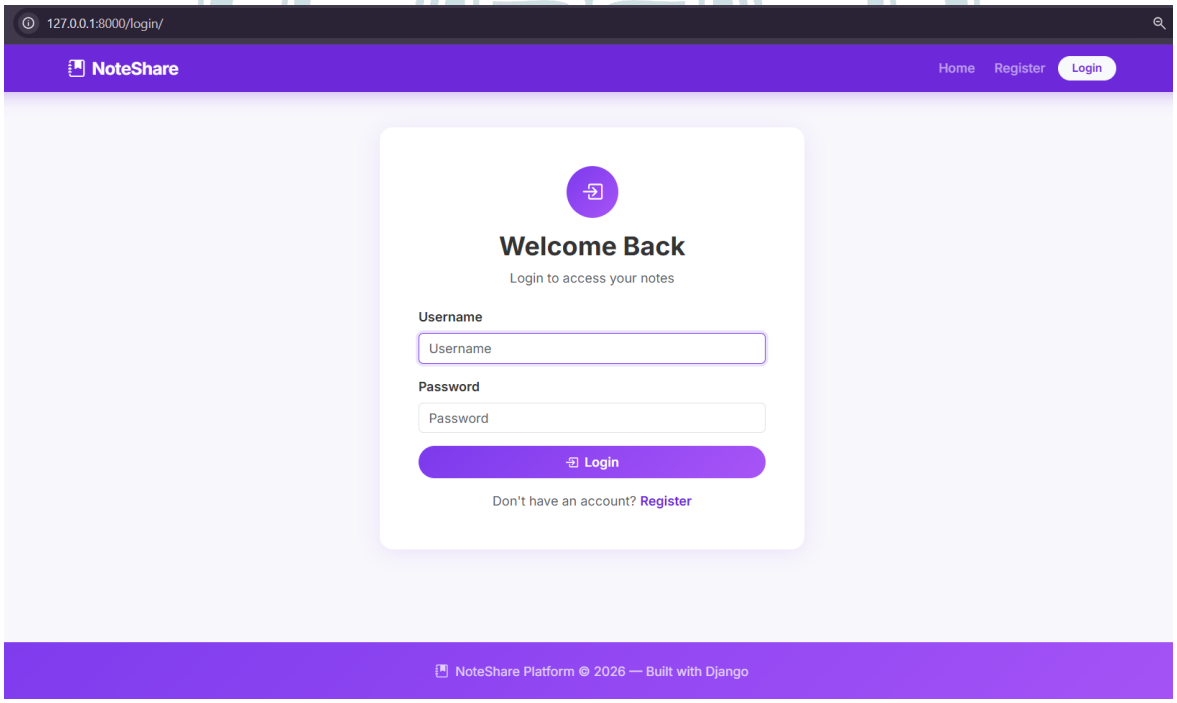
admin.py

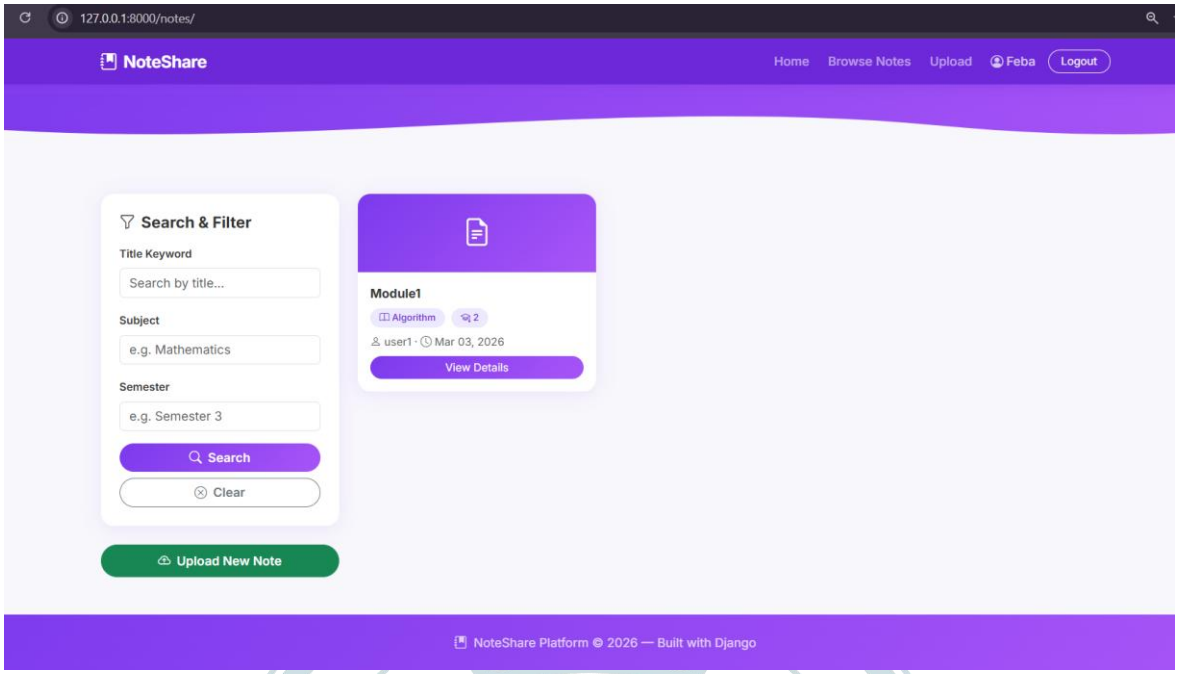
```
from django.contrib import admin
from .models import Note

@admin.register(Note)
class NoteAdmin(admin.ModelAdmin):
    list_display = ['title', 'subject', 'semester', 'uploaded_by', 'uploaded_at']
    search_fields = ['title', 'subject']
```

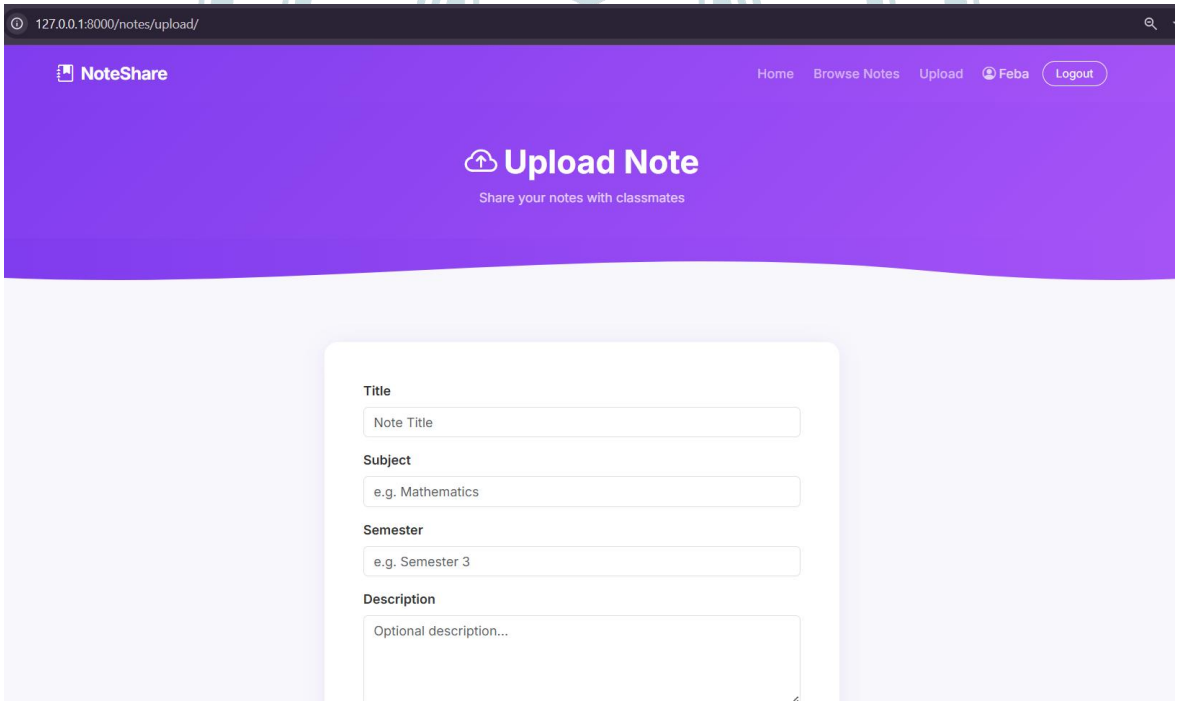
Output

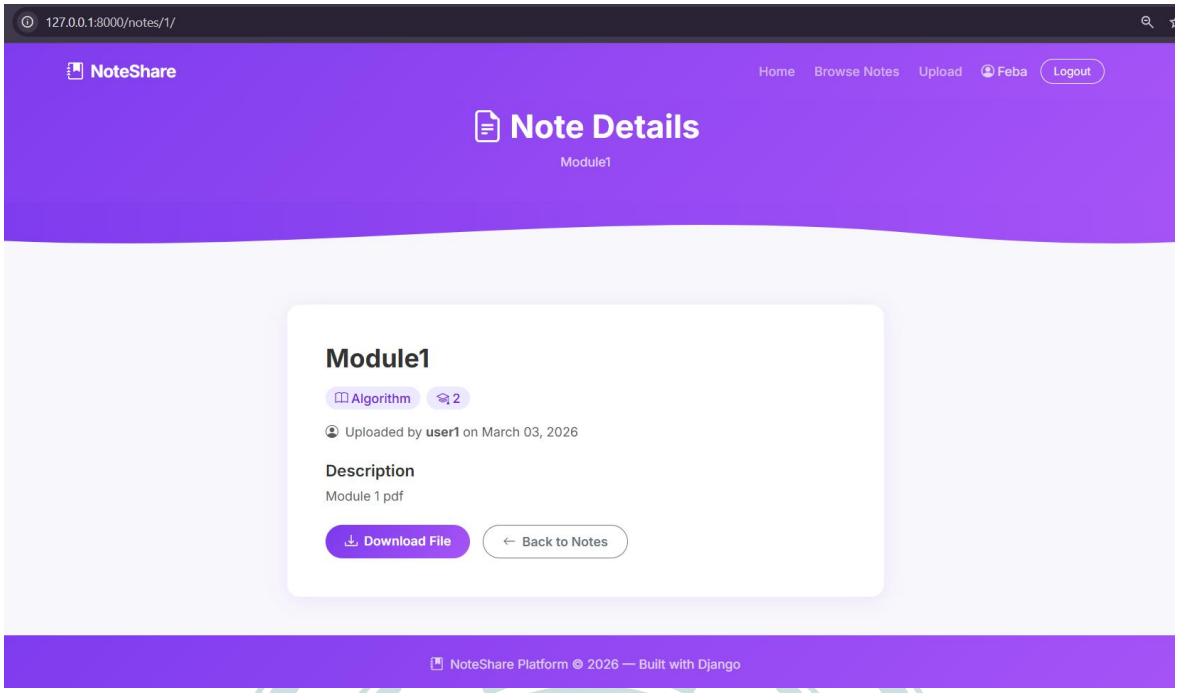

127.0.0.1:8000
NoteShare
Home
Register
Login
Share & Discover Classroom Notes
A simple platform for students to upload, browse, and download notes from every semester and subject.
Get Started
Login
How It Works
Three simple steps to get started
1. Create Account
Sign up for free and join the student community in seconds.
2. Upload Notes
Share your notes by uploading files with subject and semester tags.
3. Download & Study
Search, filter, and download notes from classmates anytime.
About Us
NoteShare is a student-driven platform designed to make sharing and accessing classroom notes effortless. Whether you missed a lecture or want to revise from a peer's perspective, NoteShare has you covered.
Our platform is simple, clean, and focused on what matters most: helping students succeed together through knowledge sharing.
NoteShare Platform © 2026 — Built with Django


127.0.0.1:8000/register/
NoteShare
Home
Register
Login
Create Account
Join NoteShare and start sharing
Username
Username
Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.
Email
Email
Password
Password
Your password can't be too similar to your other personal information.
Your password must contain at least 8 characters.
Your password can't be a commonly used password.
Your password can't be entirely numeric.
Confirm Password
Password confirmation


127.0.0.1:8000/login/
NoteShare
Home
Register
Login
Welcome Back
Login to access your notes
Username
Username
Password
Password
Login
Don't have an account? Register
NoteShare Platform © 2026 — Built with Django


127.0.0.1:8000/notes/
NoteShare
Home
Browse Notes
Upload
Feba
Logout
Search & Filter
Title Keyword
Search by title...
Subject
e.g. Mathematics
Semester
e.g. Semester 3
Search
Clear
Upload New Note
Module1
Algorithm
2
user1 · Mar 03, 2026
View Details
NoteShare Platform © 2026 — Built with Django


127.0.0.1:8000/notes/upload/
NoteShare
Home
Browse Notes
Upload
Feba
Logout
Upload Note
Share your notes with classmates
Title
Note Title
Subject
e.g. Mathematics
Semester
e.g. Semester 3
Description
Optional description...


127.0.0.1:8000/notes/1/
NoteShare
Home
Browse Notes
Upload
Feba
Logout
Note Details
Module1
Module1
Algorithm
2
Uploaded by user1 on March 03, 2026
Description
Module 1 pdf
Download File
Back to Notes
NoteShare Platform © 2026 — Built with Django